Data Structures using C & Algorithms

Data Structure :- A data structure specifies the arrangement of data in memory and set of operations that can be performed on the data present in memory.

Data structures are of 2 types

1) Primitive data structures :- Data structures / data types predefined / primitive / simple defined by a language are called primitive data structures

Eg:- int, float, double, char, long int, ------

2) Non Primitive data structures :- Data structures defined using primitive data structures are called non-primitive data structures

Eg:- array, structures, stack, queue, linked lists, trees, graphs, hashing

Non Primitive Data structures

- Linear DS :- Data is arranged in continuous memory locations logically / physically & logically.
  Eg :- array, stack, queue, structure, linked list
- Non Linear DS :- Data is arranged in hierarchical fashion (parent - child relationship)
  trees, graphs

loops & applications

arrays & applications

stack & applications

queues

linked lists & applications

stack & queue

graphs & its applications

trees, BST types & operations

hashing

searching & sorting

algorithm - time & space complexity

divide & conquer

greedy method

dynamic programming

Looping Statements / Iterative Statements / Repetitive Statements

Looping Statements are used to execute a set of statements more than once based on the output of a condition ie, the stmts inside the loop are executed if condition evaluates to true and if when condition evaluates to false the stmts following the loop are executed.

The looping stmts of C language are

1) while loop
2) do.. while loop
3) for loop.

While loop:- 1) entry controlled loop / pre-test loop

2) first condition is checked for truthfulness. If it evaluates to true, the stmts inside loop are executed.

3) Process of checking condition and executing stmts inside loop is repeated until condition evaluates to false.

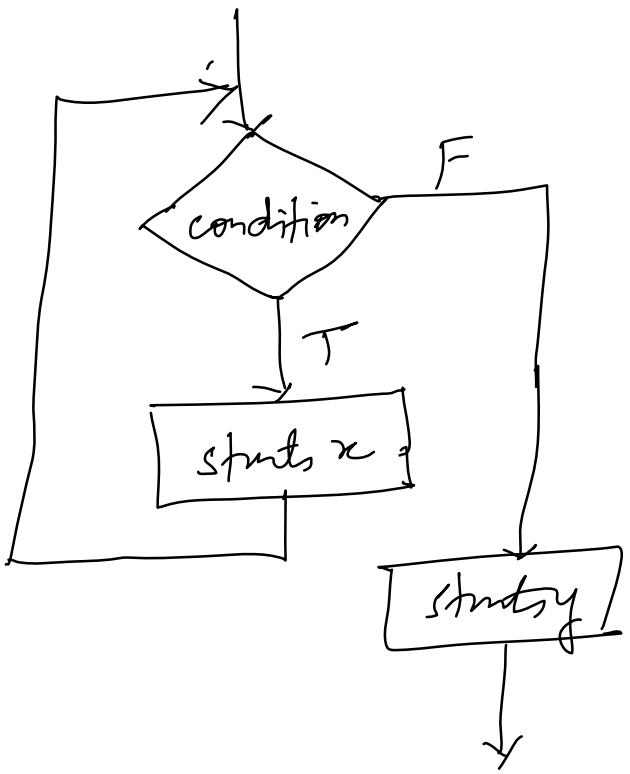
4) If condition is false for the first time itself, the stmts inside the loop are never executed.

5) while loop is generally used if we do not know the number of times the loop iterates prior to the execution of the loop.

Syntax

```
while (condition is true)
{
    statements;
}
statement;
```

Flowchart



do..while loop :- 1) exit controlled loop / post-test loop.

2) first stmts are executed and then condition is checked for truthfulness

3) Procss of executing the stmts and checking the condition for truthfulness is repeated until condition evaluates to false

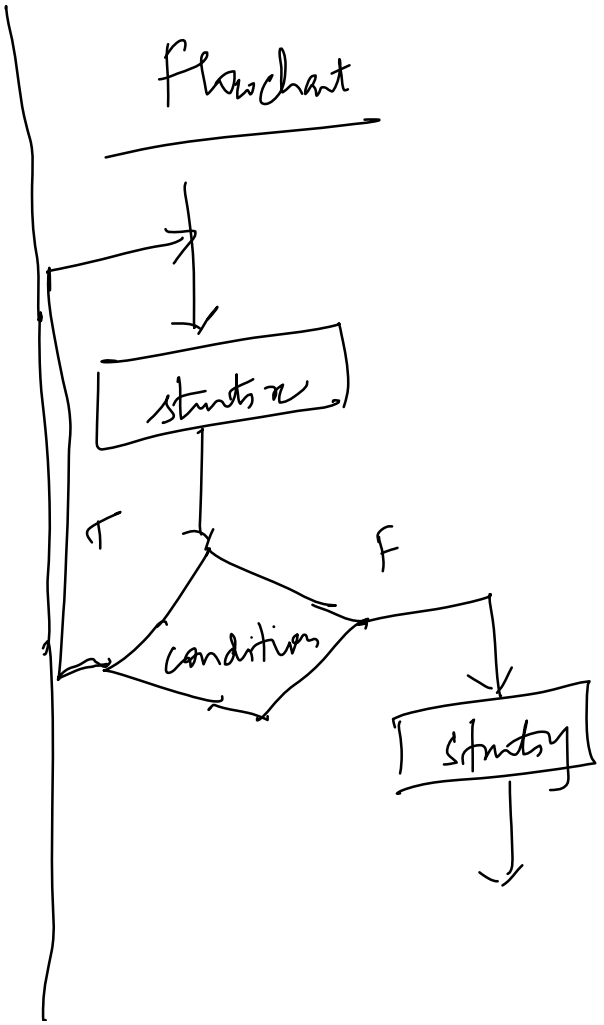
4) The stmts inside do..while loop are executed atleast once even if the condition is false for the first time itself.

5) do..while is generally used if stmts inside loop are to be executed atleast once and if we do not know the number of times the loop iterates prior to the execution of the loop.

Syntax

```
do
{
  statement x;
} while (condition is true);
statement y;
```

Flowchart



for loop:- 1) entry controlled loop / pre-test loop

2) first condition is checked for truthfulness and then if it evaluates to true the stmts inside the loop are executed.

3) Process of checking the condition and executing stmts if it evaluates to true is repeated until condition evaluates to false.

4) If the condition is false for the first time itself, the stmts inside loop are never executed.

5) for loop is generally used if we know the number of times the loop iterates prior to the execution of the loop.

Syntax

```
for(assignment ; condition ; change)
{
  stmt x ; (3)
}
stmt y ;
```

(1) (2) (4)

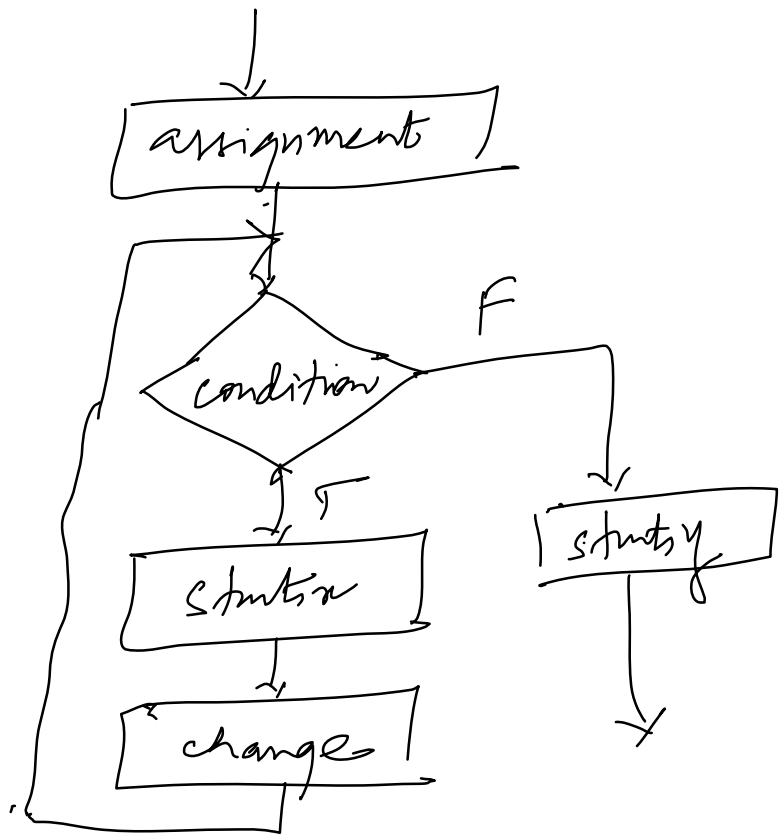
For the I time

(1) (2) (3) (4)

From II time till (2) evaluates to false

(2) (3) (4)

Flowchart



Program to find the sum of the digits of a given number :-

n

479

10) 479 (47 — 9
470
9 → %

10) 47 (4 — 7
40
7

10) 4 (0 — 4
0
4

S=0

S

0 % 10 = 0

n

9 = 479 % 10

S = S + 9 → 9

47 = 479 / 10

7 = 47 % 10

S = S + 7

4 = 47 / 10

4 = 4 % 10

S = S + 4

0 = 4 / 10

```
#include <stdio.h>
int main()
{
    int n, s=0, r;
    printf("Enter n value");
    scanf("%d", &n);
    while(n>0)
    {
        r=n%10;
        s=s+r;
        n=n/10;
    }
    printf("Sum is %d", s);
    return 0;
}
```

Program to check whether a number is armstrong or not

153 → $1^3 + 5^3 + 3^3 = 153$

$1 + 125 + 27 = 153$

$3 = 153 \% 10$

$s = s + 3 \times 3 \times 3$

$15 = 153 / 10$

$15 = 15 \% 10$

$s = s + 3 \times 5 \times 5$

$1 = 15 / 10$

$1 = 1 \% 10$

$s = s + 1 \times 1 \times 1$

$0 = 1 / 10$

m

153

n

153 15 1 0

s

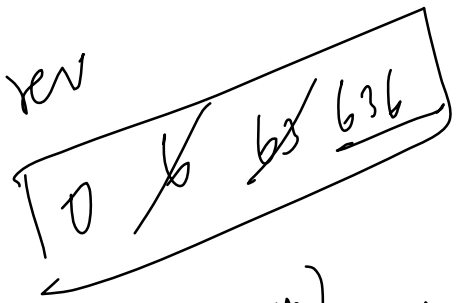
0 27 152 153

```
int main()
{
int n, m, s=0, r;

printf("Enter n value");
scanf("%d", &n);
   m=n;
while(n>0)
{
  r=n%10;
  s=s+r*r*r;
  n=n/10;
}
 if(s==m)
 printf("Armstrong");
 else
  printf("Not an armstrong");
   return 0;
}
```

Check whether a number is palindrome or not






```
do
{
    r=n%10;
    rev= rev*10+r;
    n=n/10;
} while(n>0);
if(rev==m)
printf("Palindrome");
else
printf("Not a palindrome");
```

r = 636%10

rev = rev*10+6

63 = 636/10

3 = 63%10

rev = rev*10+3

6 = 63/10

6 = 6%10

rev = rev*10+6

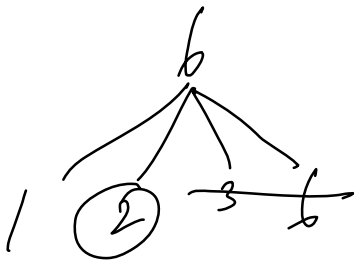
0 = 6/10

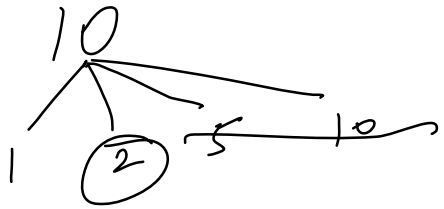
GCD of 2 values

HCF

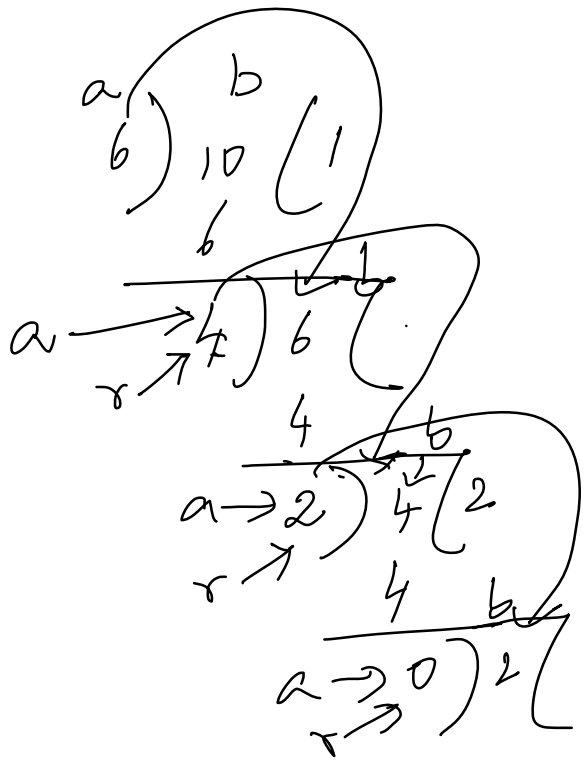
Greatest Common Divisor

Highest Common Factor

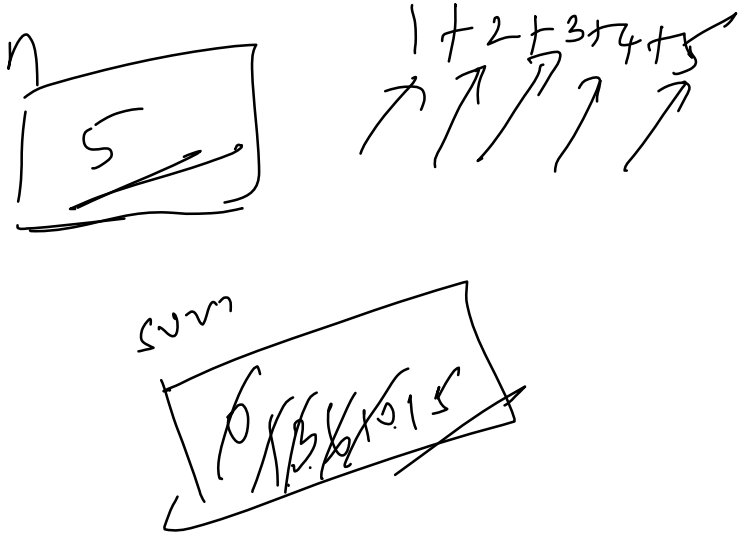


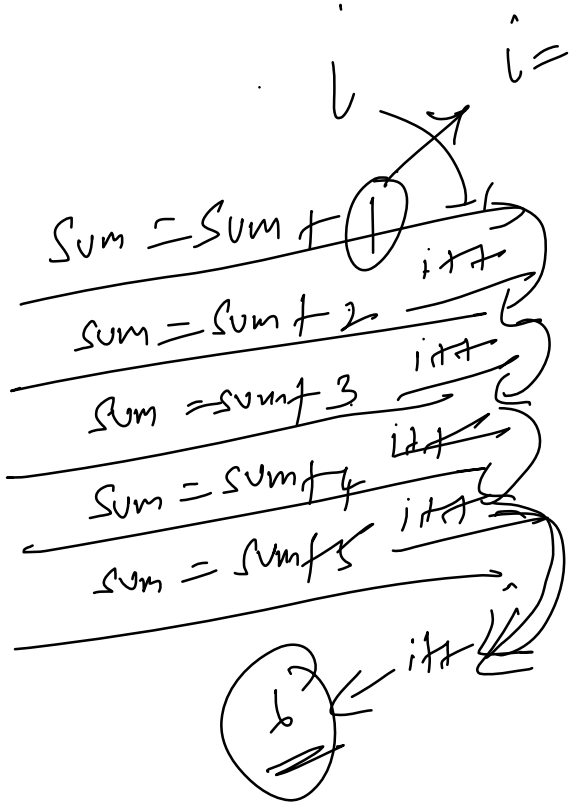


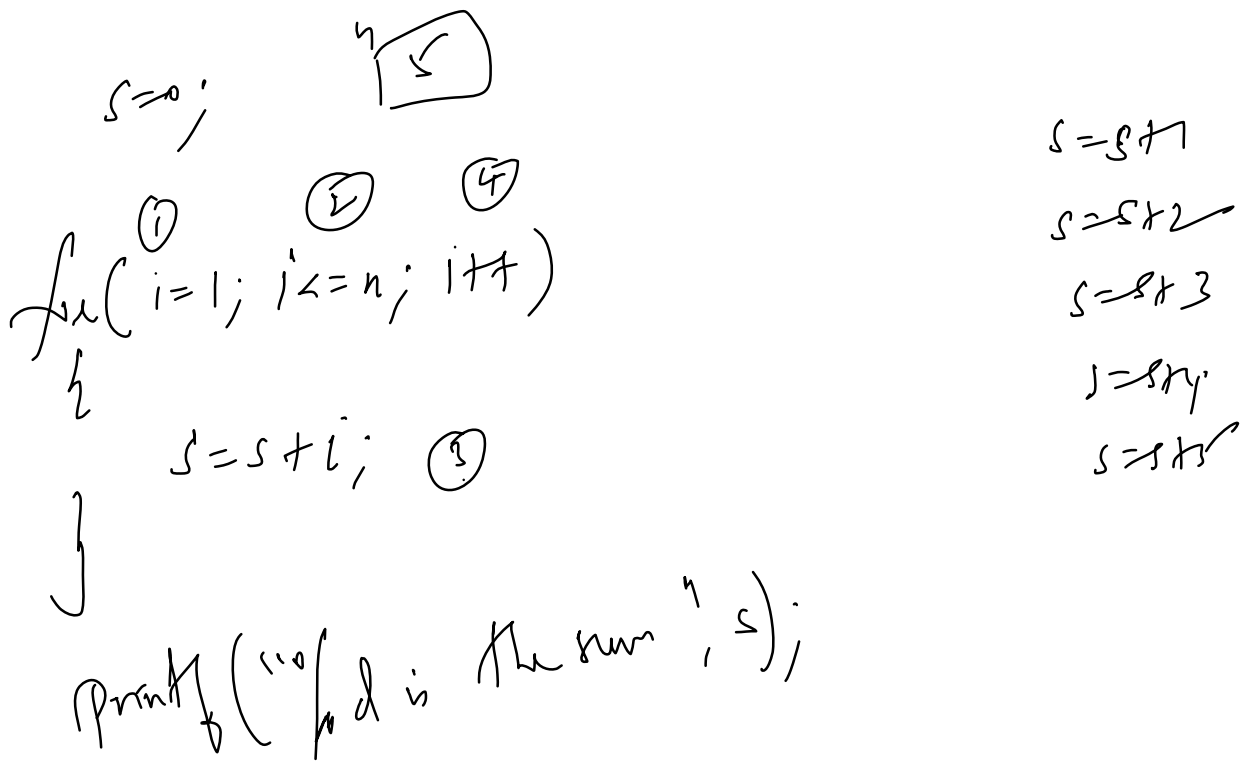
```
do
{
    r = b % a;
    b = a;
    a = r;
} while (a != 0);
printf("GCD is %d", b);
```



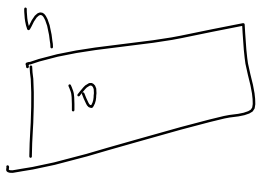
Sum of I n natural numbers





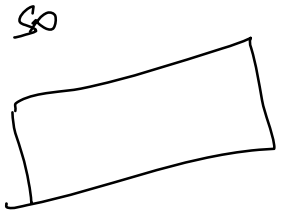

s=0;
for(i=1; i<=n; i++)
{
s=s+i;
}
printf("%d is the sum", s);
s=s+1
s=s+2
s=s+3
s=s+4
s=s+5

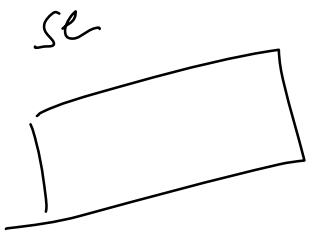
Sum of even no:'s and odd no:'s of I n natural numbers



se = 2+4+6+8+10

so = 1+3+5+7+9





1 % 2 ≠ 0	so = so + 1
2 % 2 == 0	se = se + 2
3 % 2 ≠ 0	so = so + 2
4 % 2 == 0	se = se + 4
5	so
6	se
7	so
8	se
9	so
10	se

```
so=0;
se=0;
for(i=1; i<=n; i++)
{
    if(i%2==0)
    {
        se=se+i;
    }
    else
    {
        so=so+i;
    }
}
```

Factorial of a value

n

5

1 x 2 x 3 x 4 x 5

f

120

i = 1

f = f x 1 ; i++

f = f x 2 ; i++

f = f x 3 ; i++

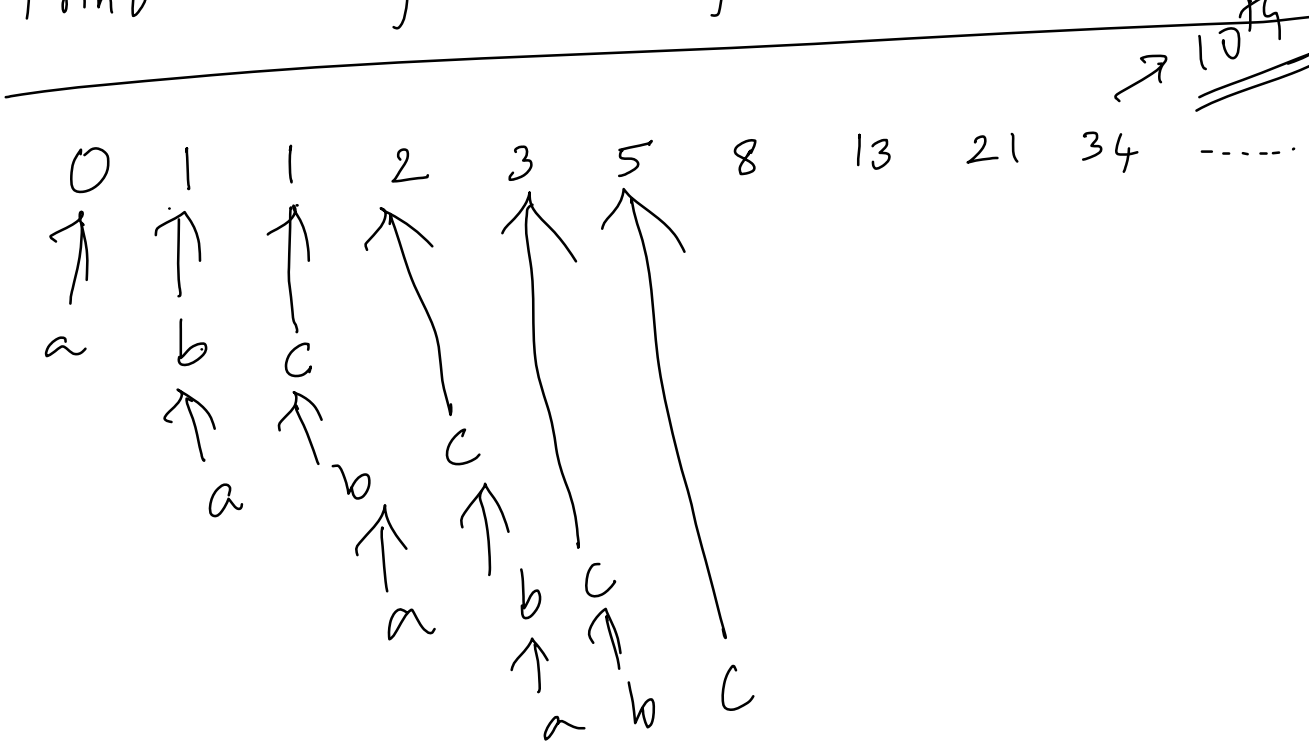
f = f x 4 ; i++

f = f x 5 ; i++

i = 6

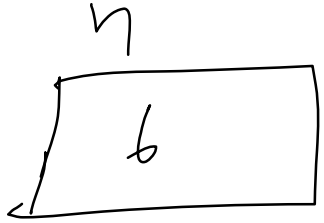
```
    f=1;
for(i=1;i<=n;i++)
{
    f=f*i;
}
```

Print the first n fibonacci numbers




```
Printf("%d %d", a, b);
for(i=3; i<=n; i++)
{
  c=a+b;
  printf("%d", c);
  a=b;
  b=c;
}
```

Print the factors of a given number



1, 2, 3, 4, 5, 6

A number when divided by its factor gives the remainder as 0.

factors are from 1 to n (given no)

n i print

if (6 % 1 == 0) → 1

if (6 % 2 == 0) → 2

if (6 % 3 == 0) → 3

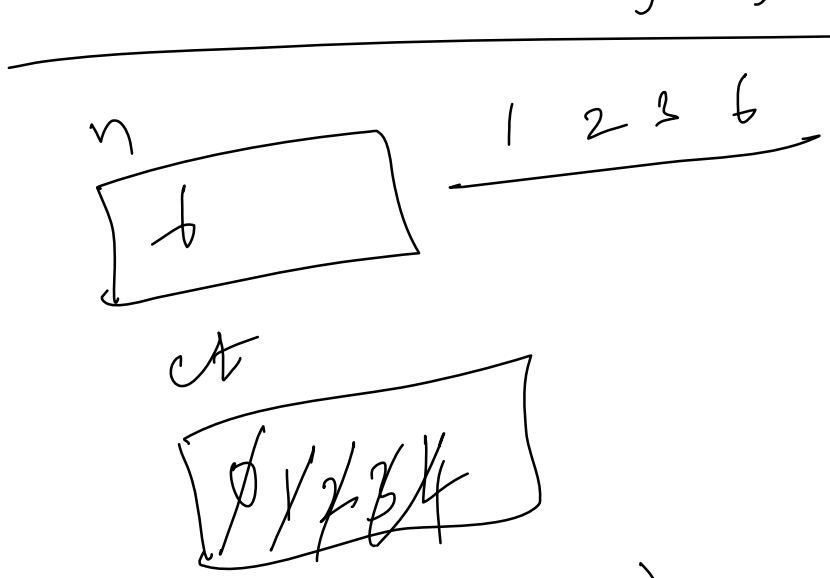
6 % 4 ≠ 0

6 % 5 ≠ 0

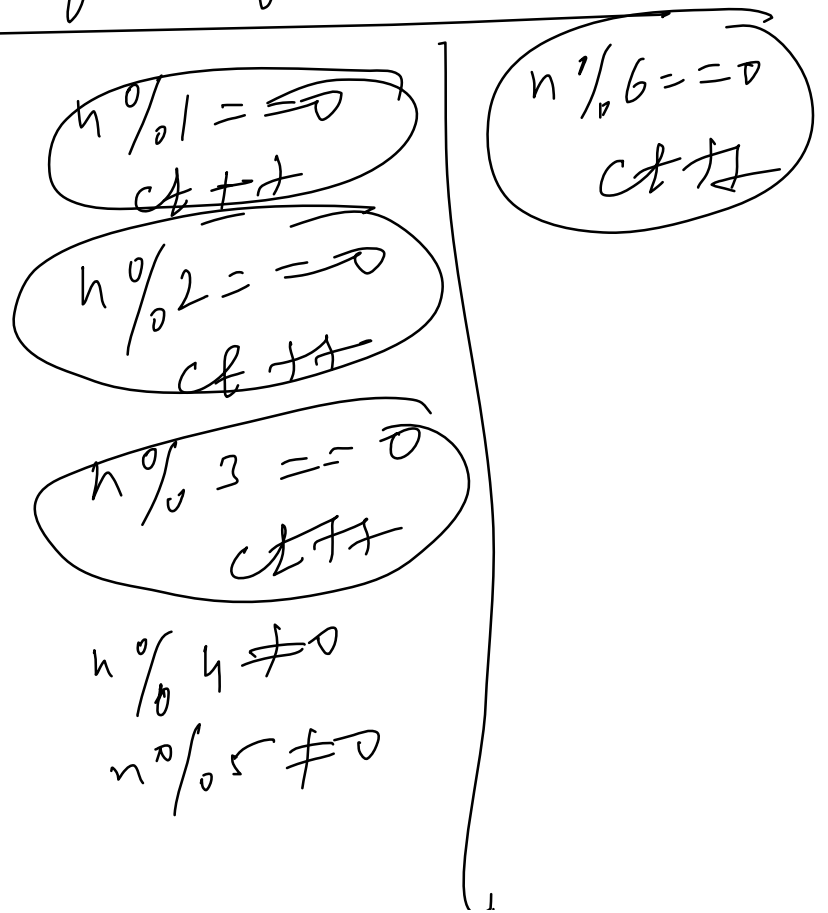
6 % 6 == 0 → 6

```
for(i=1; i<=n; i++)
{
    if(n % i == 0)
    {
        printf("%d", i);
    }
}
```

Count the no: of factors of a given value




```
for(i=1; i<=n; i++)
{   if(n%i==0)
        ct++;
}
```



Sum of the factors of a given value

1 2 3 6

n

6

S

0	3	6	12

```
for(i=1; i<=n; i++)
{
  if(n%i==0)
    s=s+i;
}
```

6%1==0
s=s+1

6%2==0
s=s+2

6%3==0
s=s+3

6%4≠0

6%5≠0

6%6==0
s=s+6

28

1 2 4 7 14 28

1
56

Check whether a given integer is a Prime or not

A number with only 2 factors

```
ct=0;
for(i=1;i<=n;i++)
{
    if(n%i==0)
    {
        ct++;
    }
}
```

```
if(ct==2)
printf("Prime");
else
printf("Composite");
```

Check whether a given number is perfect no: or not

If sum of the factors is double the given no: then it is perfect

```
s=0;
for(i=1; i<=n; i++)
{
   if(n%i==0)
      s=s+i;
}
```

```
if(s==2*n)
printf("Perfect");
else
printf("Not Perfect");
```

Break:- break stops the execution of a loop.

17

```
flag=0;
for(i=2; i<=n/2 ; i++)
{
    if(n%i==0)
    {
        flag=1;
        break;
    }
}
```

n/2 → sqrt(n)

```
if(flag==0)
printf("Prime");
else
printf("Composite");
```

50

2 3 5 7 11 13 17 19 23 29 31 37

41 43 47

```
for(n=2 ; n<=m ; n++)
{
        flag=0;
    for(i=2 ; i<=n/2 ; i++)
    {
        if(n%i==0)
        {
            flag=1;
            break;
        }
    }
    if(flag==0)
    printf("%d ",n);
}
```

```
for(n=1;n<=m;n++)
{
    s=0;
    for(i=1;i<=n;i++)
    {
        if(n%i==0)
            s=s+i;
    }
    if(s==2*n)
        printf("%d ",n);
}
```

Find the sum of the following series

$$1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots\cdots\cdots$$

$$\frac{x^1}{1!} + \frac{x^2}{2!} + \frac{x^3}{3!} + \frac{x^4}{4!} + \cdots\cdots\cdots$$

S = 1.0;

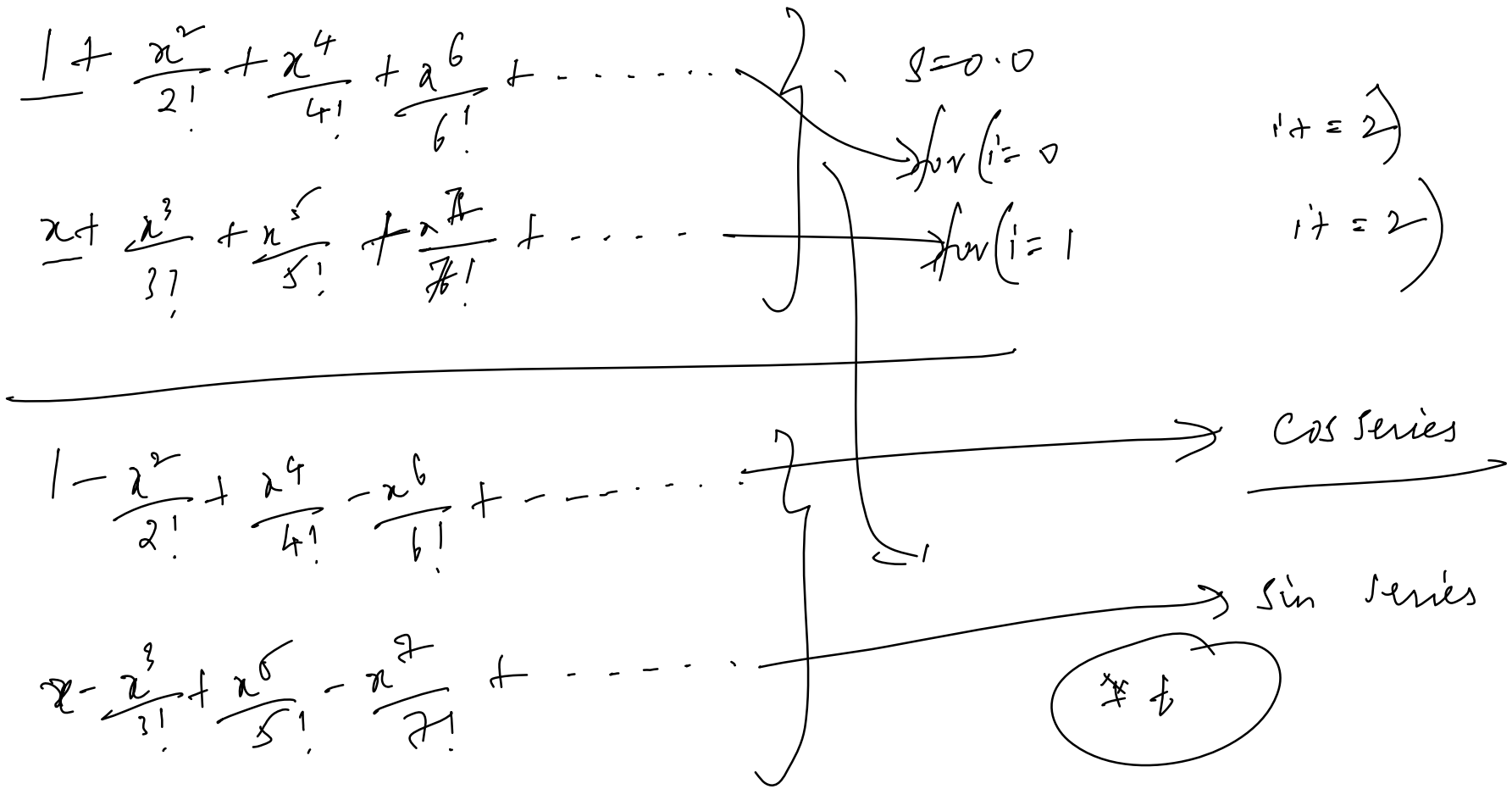
```
for(i=1; i<=n; i++)
{
S=S+ pow(x,i)/fact(i)
}
```

```
s = 1.0;
for (i=1; i<=n; i++)
{
    f = 1;
    for (j=1; j<=i; j++)
        f = f*j;
    s = s + pow(x,i)/f;
}
```

$$1 - x + \frac{x^2}{2!} - \frac{x^3}{3!} + \frac{x^4}{4!} - \frac{x^5}{5!} + \frac{x^6}{6!} - \cdots\cdots$$

$$\frac{x^0}{0!} - \frac{x^1}{1!} + \frac{x^2}{2!} - \frac{x^3}{3!}$$

```
t=1;
s=0.0;
for(i=0;i<=n;i++)
{   f=1;
    for(j=1;j<=i;j++)
      f=f*j;
    s=s+(pow(x,i)/f)*t;
    t=t*-1;
}
```


$1 + \frac{x^2}{2!} + \frac{x^4}{4!} + \frac{x^6}{6!} + \cdots\cdots$
$x + \frac{x^3}{3!} + \frac{x^5}{5!} + \frac{x^7}{7!} + \cdots\cdots$
s=0.0
for(i=0
for(i=1
i+=2)
i+=2)
$1 - \frac{x^2}{2!} + \frac{x^4}{4!} - \frac{x^6}{6!} + \cdots\cdots$
$x - \frac{x^3}{3!} + \frac{x^5}{5!} - \frac{x^7}{7!} + \cdots\cdots$
Cos Series
Sin Series
* t

Arrays:- An array is a derived datatype which is a collection of similar data items (values) stored in continuous memory locations sharing a common name.

One Dimensional Arrays

Two Dimensional Arrays.

One Dimensional Arrays (1D array) :- A 1D array is a derived datatype which is a collection of similar simple data items stored in continuous memory locations sharing a common name.

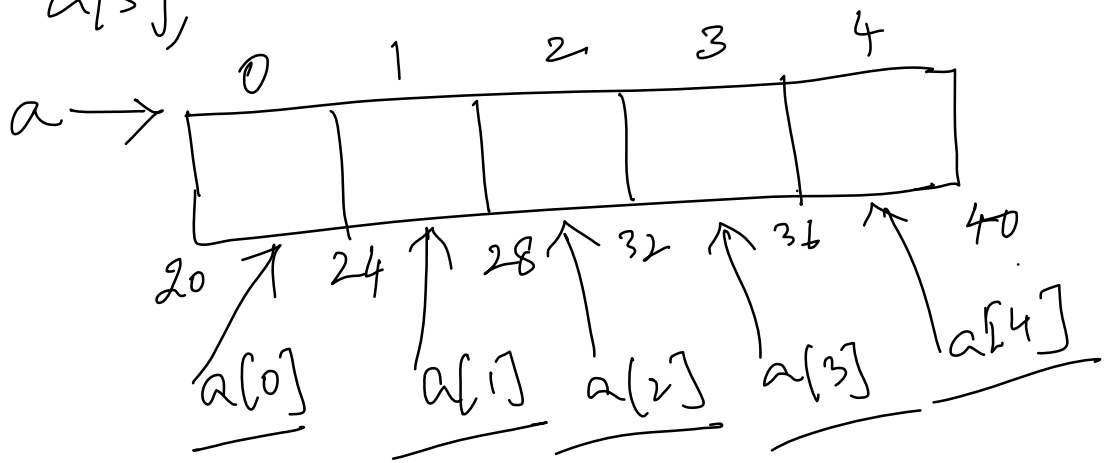
Eg :- collection of integers, collection of floats, collection of doubles etc

Syntax to declare a 1D array

datatype arrayname[size];

size → +ve integer constant > 1

Eg:- int a[5];



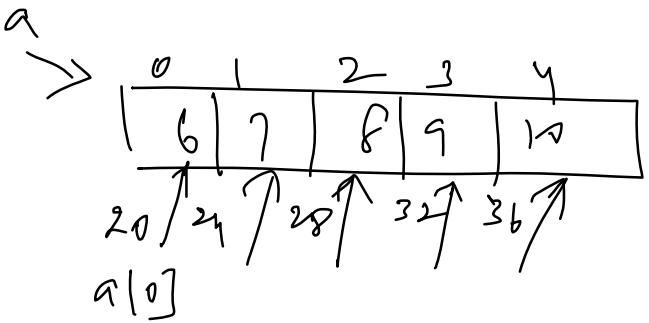
To distinguish the values of the array we use index or subscript which starts from 0

If there are n values the index of the last element is n-1

Syntax to access values present in 1D array → reading / printing / performing operations

arrayname[index]

```
int a[5];
```



Reading values

```
scanf("%d", &a[0]);
scanf("%d", &a[1]);
scanf("%d", &a[2]);
scanf("%d", &a[3]);
scanf("%d", &a[4]);
```

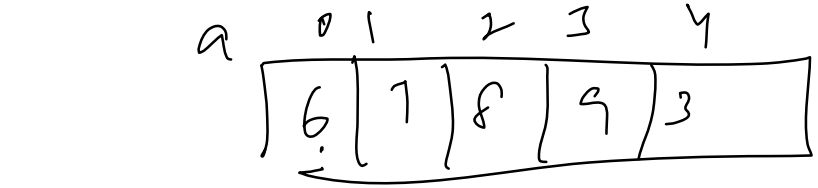
```
for(i=0; i<5; i++)
{
scanf("%d", &a[i]);
}
```

Printing the values

```
for(i=0; i<5; i++)
{
printf("%d", a[i]);
}
```

1) Sum of the elements of the array
2) Sum of odds and even elements of the array
3) Maximum element in an array ✓
4) Minimum element in an array ✓
5) Search for an element in an array
6) Second maximum element of an array ✓
7) First fixed point of an array ✓
8) Find the no: of times an element has occured in an array.

```
int a[100], i, n, s
printf("Enter n");
scanf("%d", &n);
for(i=0; i<n; i++)
{ scanf("%d", &a[i]);
}   s=0;
for(i=0; i<n; i++)
{
   s = sta[i];
}
printf("Sum is %d", s);
```



s

s = sta[0]

s = sta[1]

s = sta[2]

s = sta[3]

s = sta[4]

```
se=0;  so=0;
for(i=0; i<n; i++)
{
    if(a[i]%2==0)
        se=se+a[i];
    else
        so=so+a[i];
}
printf("%d %d", se, so);
```

0	1	2	3	4	5	6	7	8	9
2	7	3	5	9	4	8	1	6	10

a[0]%2==0 se+=a[0]

a[1]%2≠0 so+=a[1]

a[2]%2≠0 so+=a[2]

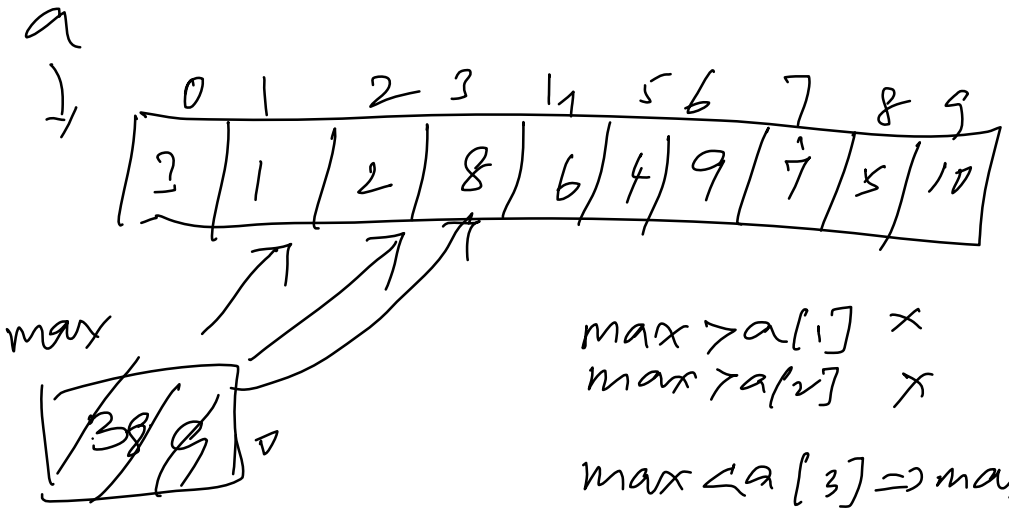
a[3]%2≠0 so+=a[3]

a[4]%2≠0 so+=a[4]

⋮

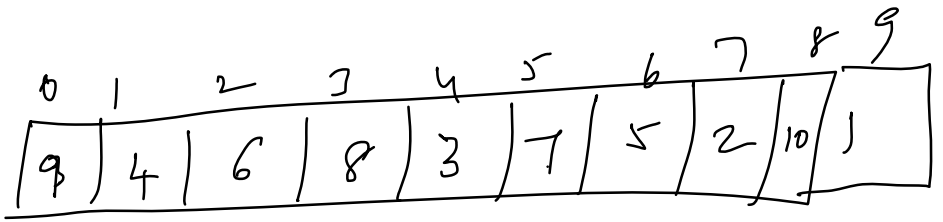
a[9]%2==0 se+=a[9]

```
max=a[0];
for(i=0;i<n;i++)
{
if(max<a[i])
{
  max=a[i];
}
}
```

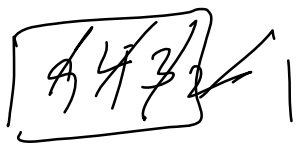


max > a[1] ×
max > a[2] ×
max < a[3] ⇒ max = a[3]
~~max > a[4] ×~~
max > a[5] ×
max < a[6] ⇒ max = a[6]
max > a[7]
max > a[8]
max < a[9] ⇒ max = a[9]

```
min = a[0]
for(i=1; i<10; i++)
{
  if(min > a[i])
    min = a[i];
}
```



min



min > a[1] ⇒ min = a[1]

min < a[2]

min < a[3]

min > a[4] ⇒ min = a[4]

min < a[5]

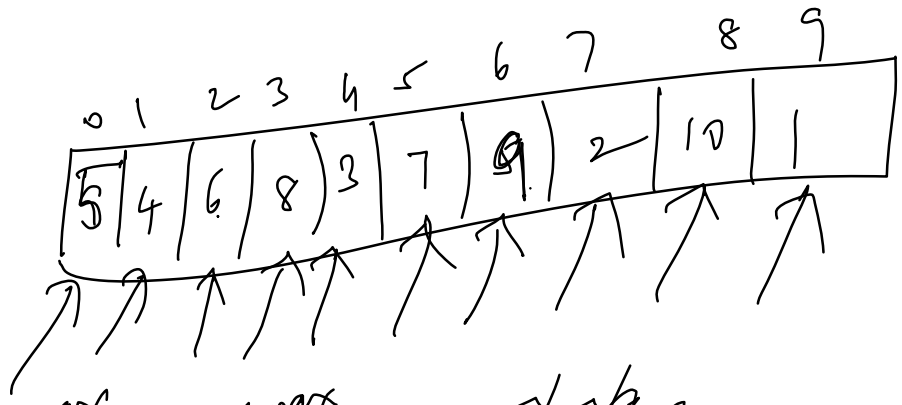
min < a[6]

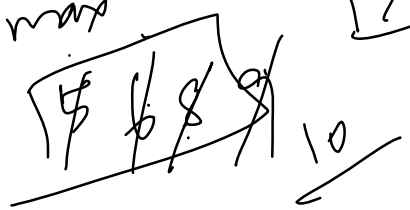
min > a[7] ⇒ min = a[7]

min < a[8]

min > a[9] ⇒ min = a[9]

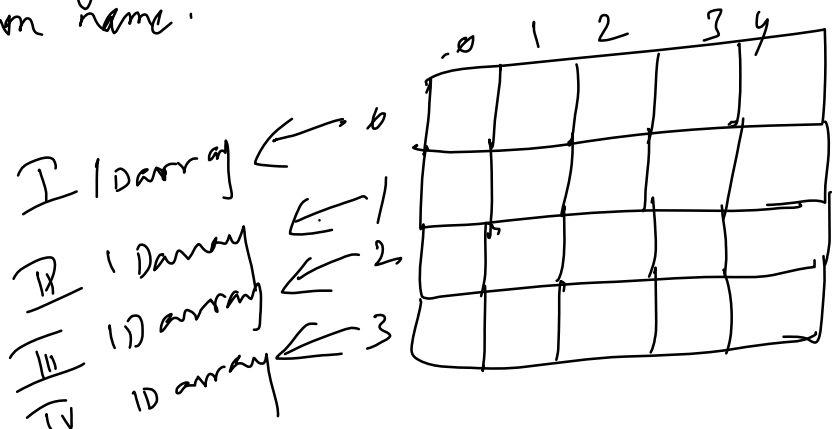
```
max = a[0];
smax = 0;
if (max < a[i])
{
    smax = max;
    max = a[i];
}
else if (smax < a[i])
    smax = a[i];
}
printf("%d", smax);
```





2 Dimensional Arrays :-

A 2D array is a collection of 1D arrays stored in continuous memory locations sharing a common name.



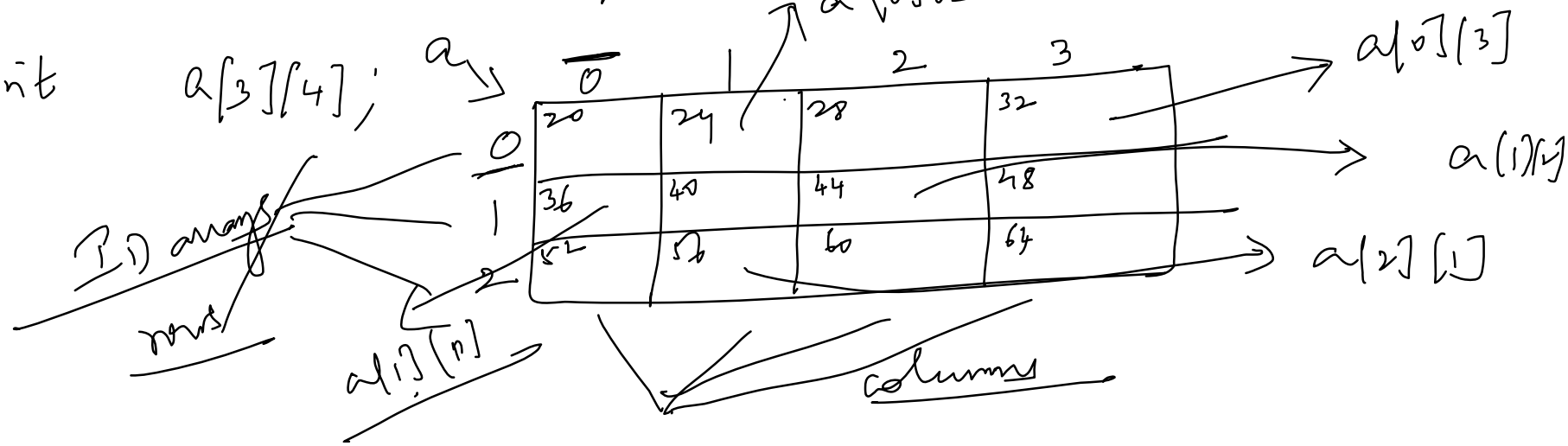
Syntax to declare a 2D array

```
datatype arrayname[rowsize][colsize];
```

rowsize: no. of 1D arrays

colsize: no. of values in each 1D array

int a[3][4];



Syntax to access the values of a 2D array

To distinguish values of 2D array we use rowindex and column index which start from 0

```
arrayname[rowindex][colindex]
```

Inputs

```
scanf("%d", &a[0][0]);
scanf("%d", &a[0][1]);
scanf("%d", &a[0][2]);
scanf("%d", &a[0][3]);
```

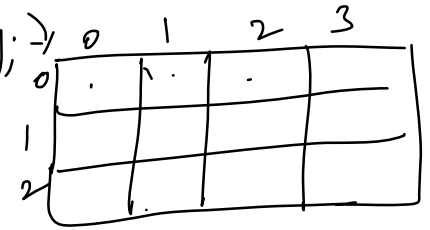
Reading

```
for(j=0; j<4; j++)
{
  scanf("%d", &a[0][j]);
}
for(j=0; j<4; j++)
{
  scanf("%d", &a[1][j]);
}
for(j=0; j<4; j++)
{
  scanf("%d", &a[2][j]);
}
```

```
for(i=0; i<3; i++)
{
  for(j=0; j<4; j++)
  {
    scanf("%d", &a[i][j]);
  }
}
```

```
int a[3][4];
```

a



Printing

```
for(i=0; i<3; i++)
{
    for(j=0; j<4; j++)
    {
        printf("%d ", a[i][j]);
    }
    printf("\n");
}
```

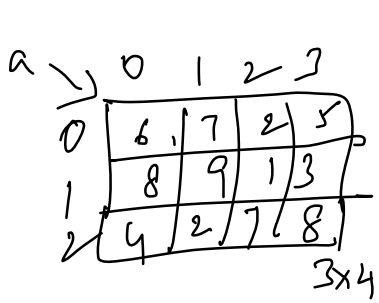
1) Adding of 2 matrices
2) Multiplication of 2 matrices
3) Transpose of a matrix
4) Symmetric matrix or not
5) Upper Δ^{lar} matrix
6) Lower Δ^{lar} matrix
7) Trace of a matrix
8) Determinant of a 3×3 matrix

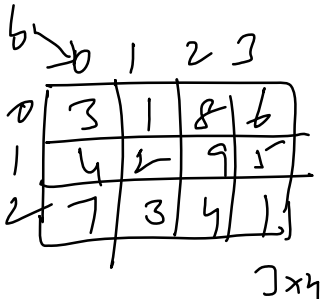
Add 2 matrices :-

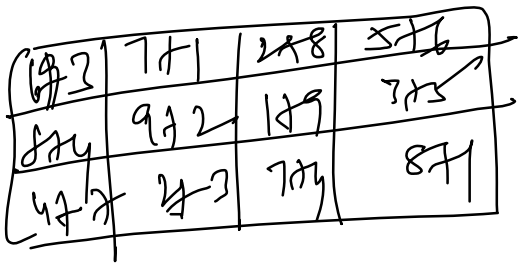
a[0][0] + b[0][0]
a[0][1] + b[0][1]
a[0][2] + b[0][2]
a[0][3] + b[0][3]

```
for(i=0; i<3; i++)
{
for(j=0; j<4; j++)
{
c[i][j] = a[i][j] + b[i][j];
}
}
```

Both the matrices must be same order







Multiply 2 matrices :- no: of cols of I matrix = no: of rows of II matrix

3×4 4×2 o/p:- no: of rows of I matrix × no: of cols of II matrix

a →

	0	1	2	3
0	1	2	3	4
1	5	6	7	8
2	3	9	1	2

3×4

b →

	0	1
0	3	7
1	1	4
2	6	2
3	8	5

4×2

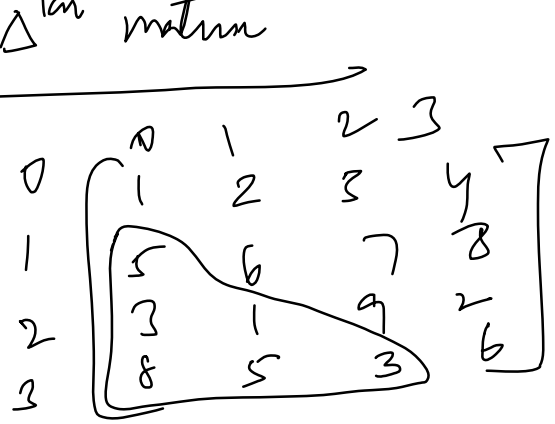
i j i k k j

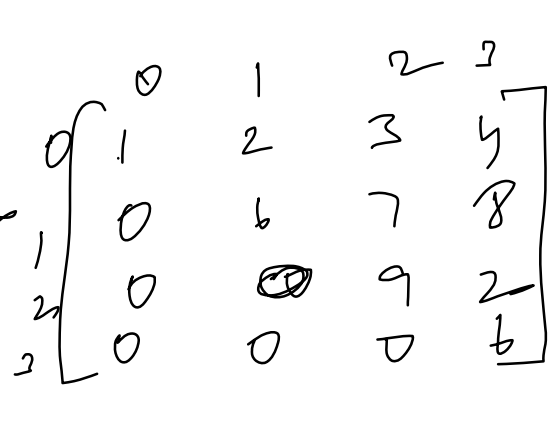
c[i][j] = a[i][0] * b[0][j]
+ a[i][1] * b[1][j]
+ a[i][2] * b[2][j]
+ a[i][3] * b[3][j]

c[0][0] = 1×3 + 2×1 + 3×6 + 4×8

```
for(i=0; i<3; i++)
{
  for(j=0; j<2; j++)
  {
    c[i][j] = 0;
    for(k=0; k<4; k++)
      c[i][j] = c[i][j] + a[i][k] * b[k][j];
  }
}
```

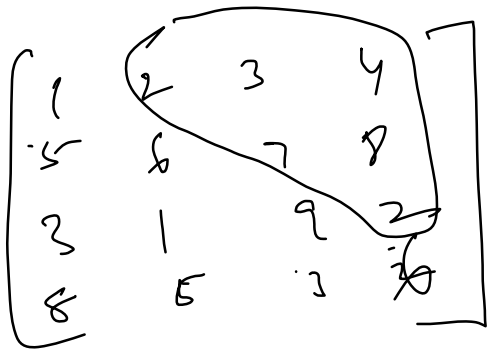
Upper Δ^lar matrix

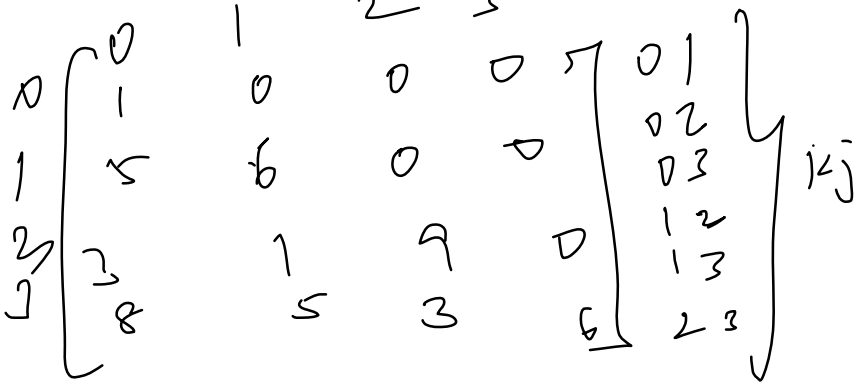




i > j

Lower Δ^lar matrix





ikj

Upper Δlar matrix

```
for(i=0; i<m; i++)
{
  for(j=0; j<n; j++)
  {
    if(i>j)
      a[i][j]=0;
  }
}
```

Lower Δlar matrix

```
for(i=0; i<m; i++)
{
  for(j=0; j<n; j++)
  {
    if(i<j)
      a[i][j]=0;
  }
}
```

$$\begin{matrix} & 0 & 1 & 2 \\ 0 & 1 & 2 & 3 \\ 1 & 4 & 5 & 6 \\ 2 & 7 & 8 & 9 \end{matrix}$$

$1(5\times9-8\times6)-2(9\times4-7\times6)+3(8\times4-7\times5)$

$=1(5\times9-8\times6)+2(7\times6-9\times4)+3(8\times4-7\times5)$

$=a[0][0]*(a[1][1]*a[2][2]-a[2][1]*a[1][2])$

$+a[0][1]*(a[2][0]*a[1][2]-a[2][2]*a[1][0])$

$+a[0][2]*(a[2][1]*a[1][0]-a[2][0]*a[1][1])$

~~for(j=0;j<3;j++)~~

$det+=a[i][j]*(a[i+1\%3][j+1\%3]*a[i+2\%3][j+2\%3]-a[i+2\%3][j+1\%3]*a[i+1\%3][j+2\%3])$

}

String:- It is a collection of characters stored in a character array ending with a null ('\0') character

Read Strings

scanf("%s", name of characterarray);
reads single word strings

gets(name of characterarray);
reads single line strings

Print Strings

printf("%s", name of characterarray);
prints the string

puts(name of characterarray);
Prints the string with an enter key.

String Handling Functions

1) int strlen(char s[]) :- length of the string

2) char* strcpy(char dst[], char src[]) :- copies string in src to dest.

3) char* strrev(char s[]) :- reverses a string

4) char* strcat(char s1[], char s2[]) :- adds string present in s2 at end of s1

5) int strcmp(char s1[], char s2[]) :- compares 2 strings and returns 0 if they are equal, s1>s2 returns 1, s1<s2 returns -1

6) char* strlwr(char s[]) :- converts a string to lower case string

7) char* strupr(char s[]):- converts a string to upper case string

8) char* strset(char s[], char ch):- replaces every character in s[] with ch.

9) char* strchr(char s[], char ch):- returns the first occurance of ch in s

10) char* strstr(char s1[], char s2[]):- returns the first occurance of s2 in s1

Structures :- A structure is a user defined datatype which is a collection of dissimilar data items stored in continuous memory locations sharing a common name.

A structure must be defined first and then the variables of the structure must be declared and used.

The keyword useful to work with structures is

struct

Syntax to define a structure

```
struct tagname
{
    datatype membervariable1;
    datatype membervariable2;
    :
};
```

Syntax to declare a structure variable

struct tagname structvariable1, structvariable2, ----;

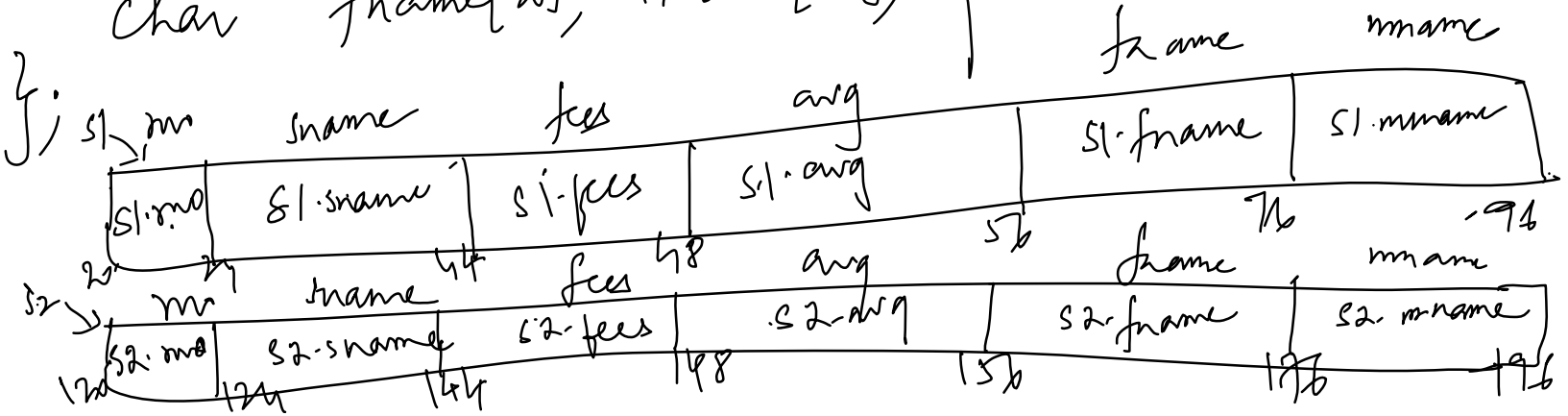
Eg:-

definition

```
struct student
{
    int rno;
    char sname[20];
    float fees;
    double avg;
    char fname[20], mname[20];
};
```

declaration

```
struct student s1, s2;
```



To access the members of a structure or differentiate the members of the structure we use structure variable name and a "." operator

Syntax →

structure variable . structure member

Pointer :- A pointer is a variable which stores the address of another variable of its datatype.

Syntax to declare a pointer

datatype *pointervariablename;

→ pointer declaration operator

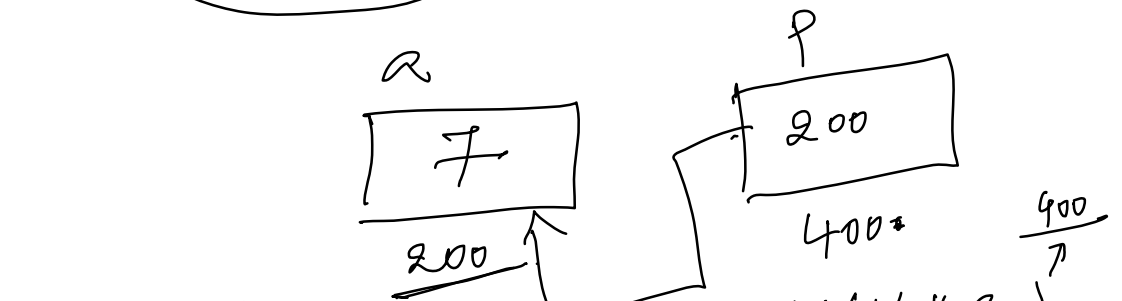
Eg:- int *p;

To store the address of a variable we use the operator **&**.

Syntax to store the address of a variable inside a pointer variable is

pointervariablename = &normalvariablename;

Eg:- int *p;
int a=7;
p=&a;



printf("%d", a); → 7
printf("%u", &a); → 200
printf("%u", p); → 200
printf("%u", &p); → 400
printf("%d", *p);
printf("%d", *(&a));

Addresses are always +ve integers. Hence we use %u when we are working with pointers & addresses.

To access the value present in a variable using the pointer variable we use the operator *
and the syntax is *pointervariablename

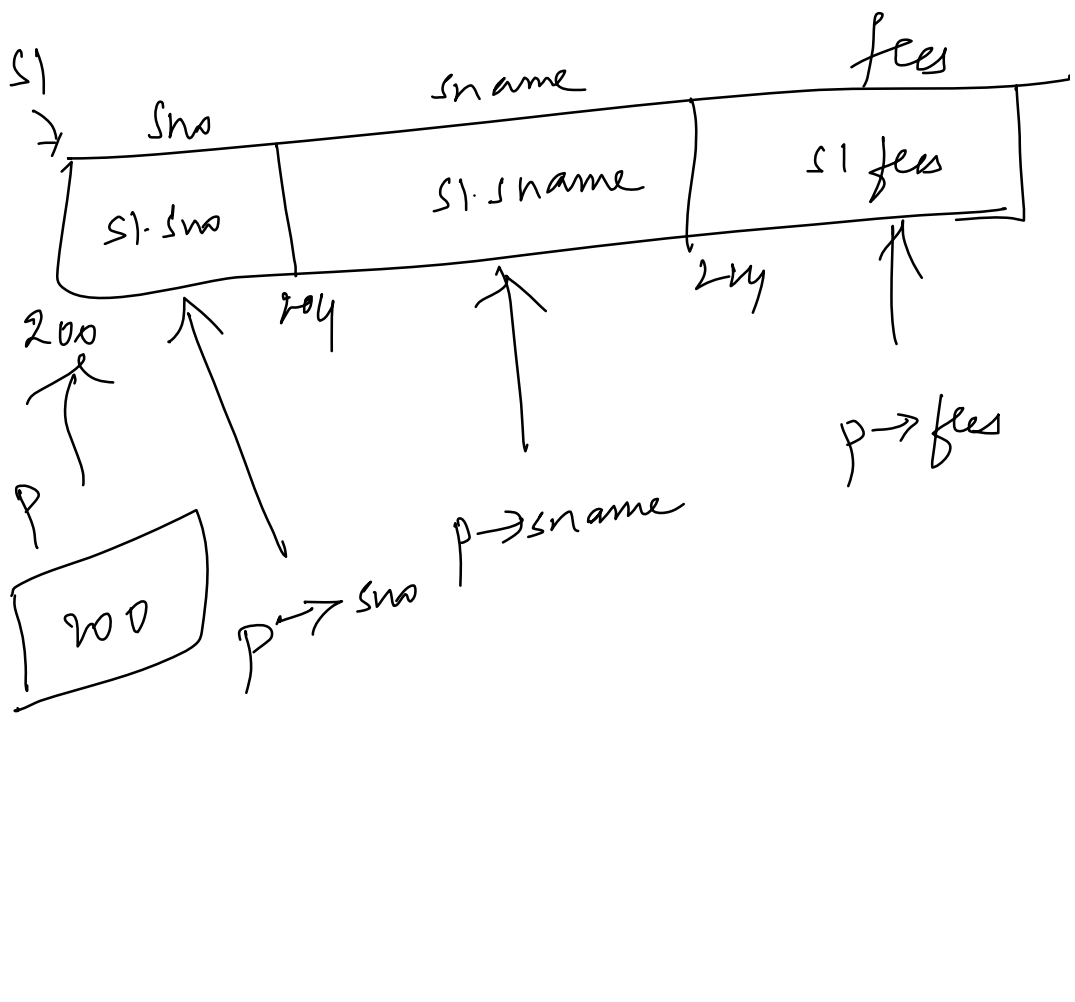
value at address of operator

* and & are inverse operators

```
struct student
{
    int sno;
    char sname[20];
    float fees;
};

struct student s1, *p;

p = &s1;
```



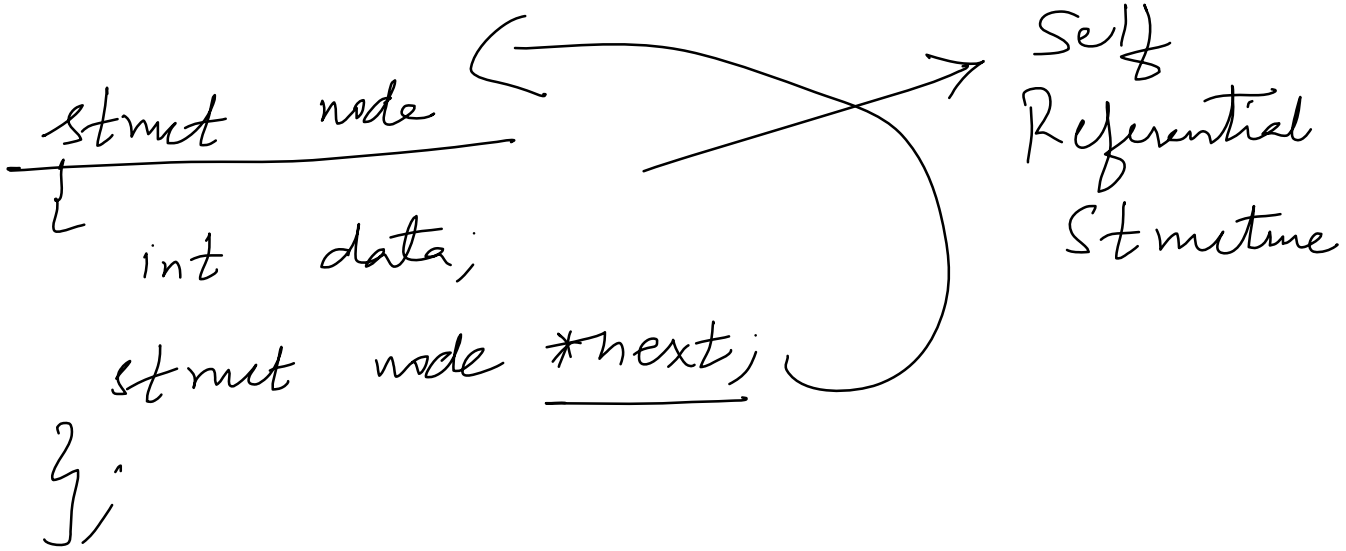
To access members of the structure using structure pointer we use the "→" operator

---

A structure can be defined or declared inside another structure. Such structure is called nested structure.

We can also declare a structure pointer inside the structure

For eg:-


struct node
{
int data;
struct node *next;
};
Self
Referential
Structure

Stack:- It is a linear data structure which has a special property of **Last In First Out**. It is also LIFO data structure.

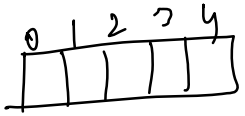
Operations on a Stack

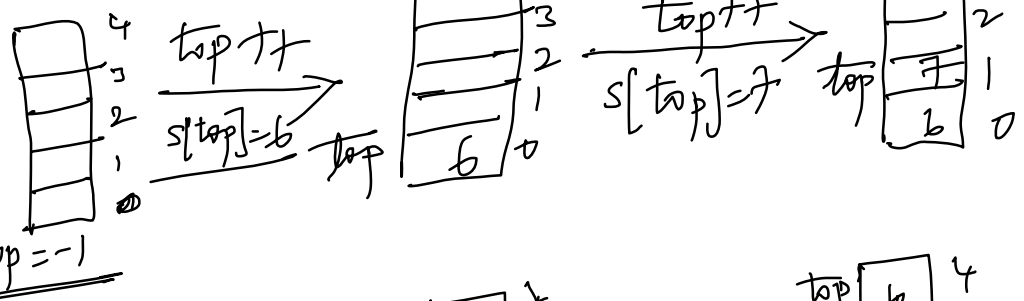
1) Push :- Placing an element into top most position of stack
2) Pop :- Removing an element from top most position of stack
3) Display :- Displaying the elements from top most position to first position of stack.

Implementation of a stack using an array

Push Operation

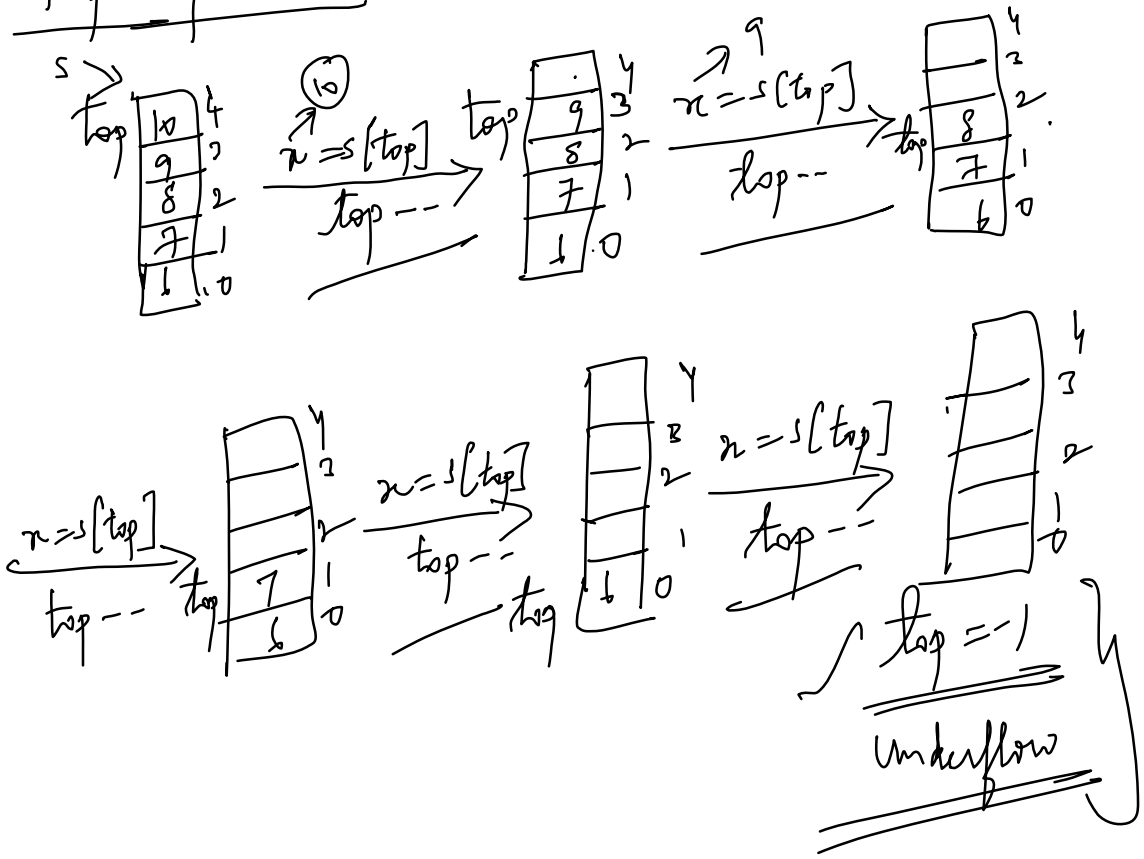
size = 5 , s[5]






```
Void push (int x)
{
  if (top == size-1)
  {
    printf("Stack Overflow");
    return;
  }
  top++;
  s[top] = x;
}
```

Pop Operation




```
int pop()
{
    int x;
    if(top==-1)
    {
        printf("Stack Underflow");
        return -1;
    }
    x=s[top];
    top--;
    return x;
}
```

Display :-

```
void display()
{
  int i;
  if(top == -1)
  {
    printf("stack is empty");
      return;
  }
  for(i=top ; i>=0 ; i--)
  {
    printf("%d ", s[i]);
  }
}
```

Linear Queue / Simple Queue:-

A Queue is a linear data structure which has a special property of First-In-First-Out.

Operations

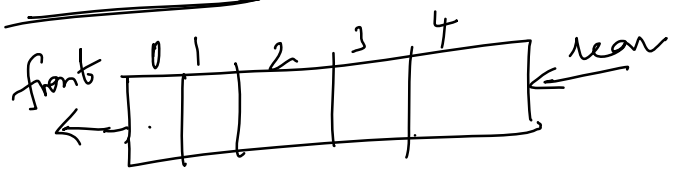
1) **Insertion**:- Placing a value into queue from rear position of queue
2) **Deletion**:- Deleting a value from the queue front position of queue
3) **Display**:- Printing the values of the queue from front to rear position

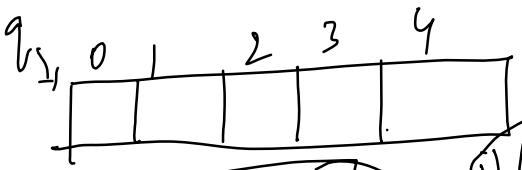
Implementation of a Queue using an array

q[size], size = 5

r = -1, f = -1

Insertion

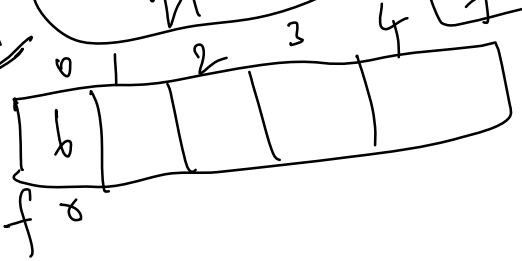




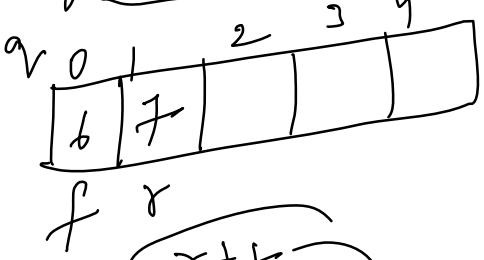
r = -1
f = -1

r++
q[r] = 6

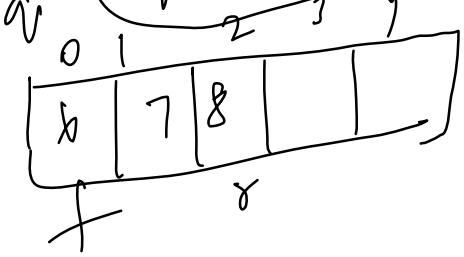
if(f == -1)
f = 0



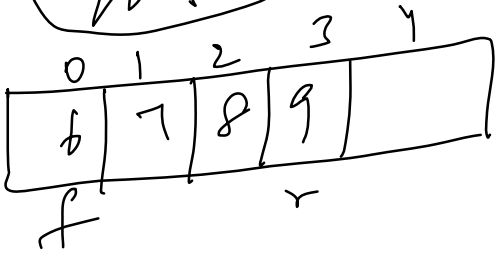
r++
q[r] = 7



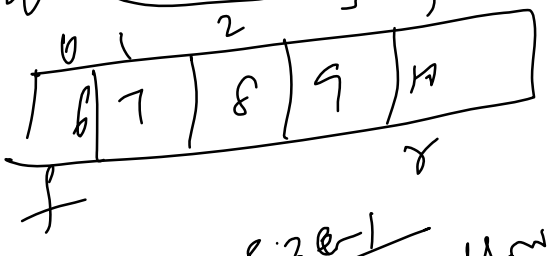
r++
q[r] = 8



r++
q[r] = 9



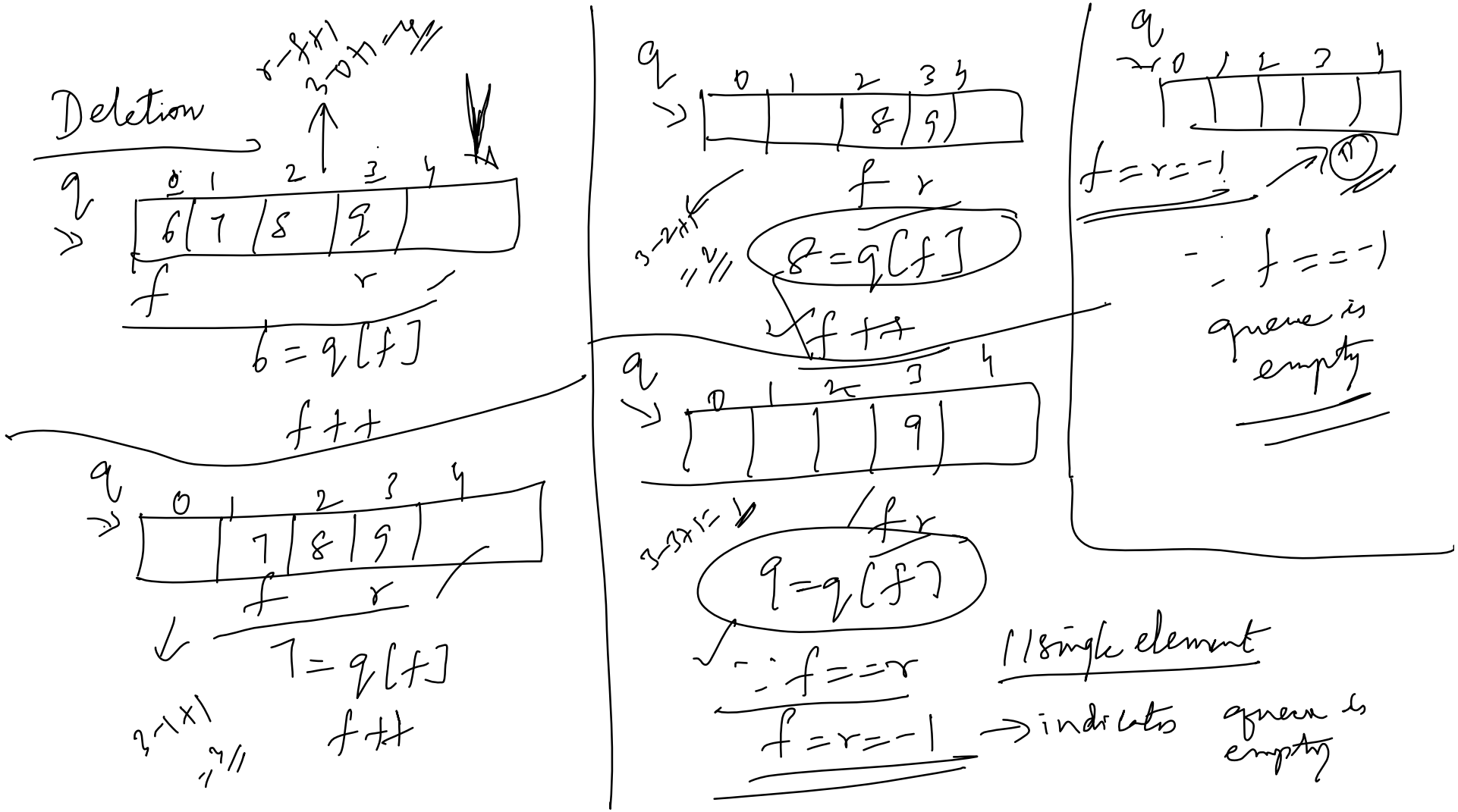
r++
q[r] = 10



r == size-1
⇒ queue is overflow

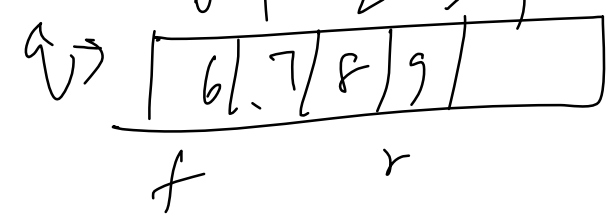
```
void qinsert (int x)
{
  if (r==size-1)
  {
    printf ("Queue overflow");
      return;
  }

      r++;
      q[r]= x;
      if (f==-1)
        f=0;
}
```


Deletion
q
0 1 2 3 4
6 7 8 9
f r
b = q[f]
f++
7 8 9
f r
7 = q[f]
f++
8 9
f r
8 = q[f]
f++
9
f r
9 = q[f]
// single element
f == r
f == r == -1 → indicates queue is empty
f == r == -1
queue is empty

```
int qdelete()
{
    int x;
    if(f==-1)
    {
        printf("Queue is empty");
        return -1;
    }
    x=q[f];
    if(f==r)          //single element condition
        f=r=-1;
    else
        f++;
    return x;
}
```

Display




```
void qdisplay()
{
  int i;
  if(f==-1)
  {
    printf("Queue is empty\n");
    return;
  }
  for(i=f;i<=r;i++)
  {
    printf("%d ",q[i]);
  }
  printf("\n");
}
```

Circular Queue :- A circular queue is a queue where insertions and deletions are allowed in circular fashion.

Operations :-

Insertion :- Placing a value into the queue from rear position of the queue.

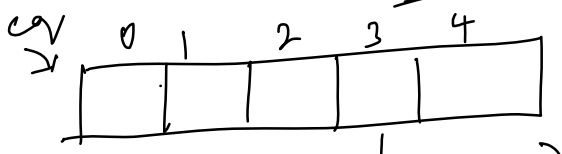
Deletion :- Removing a value from the queue from front position of queue

Display :- Printing values from front position to rear position of queue.

Insertion

cq[5]

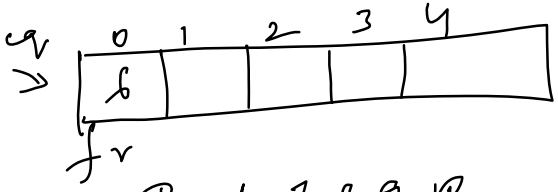
r=-1, f=-1



r=-1
f=-1

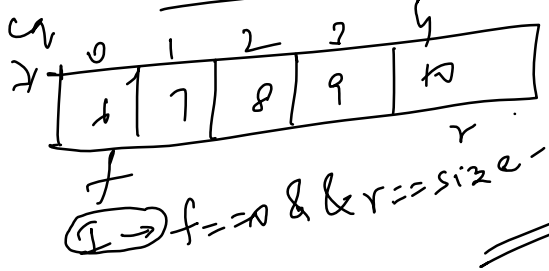
r++
cq[r]=6

if(f==-1)
f=0



Insert 7, 8, 9, 10

r++ cq[r]=x



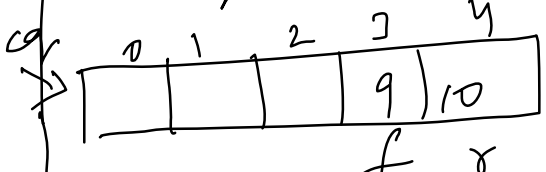
① → f==0 && r==size-1 ✓

3 deletions

6 = cq[f], f++

7 = cq[f], f++

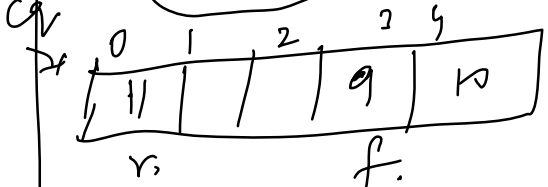
8 = cq[f], f++



∴ r==size-1

r=0, cq[r]=x

11

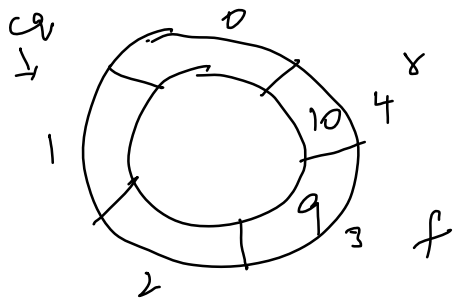


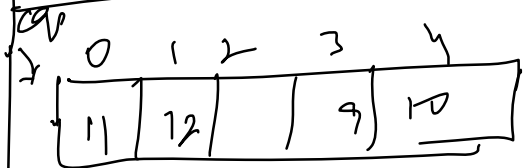
r++

cq[r]=x

12

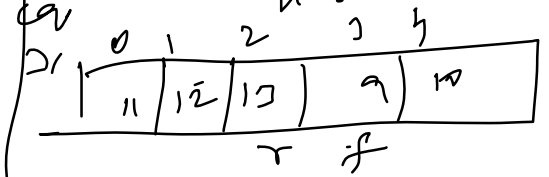
② → f==r+1





r++

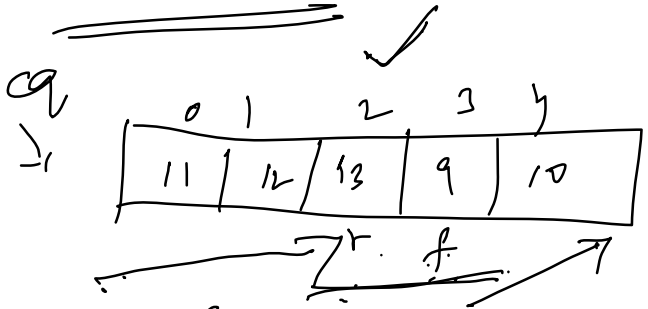
cq[r]=13



② → f==r+1

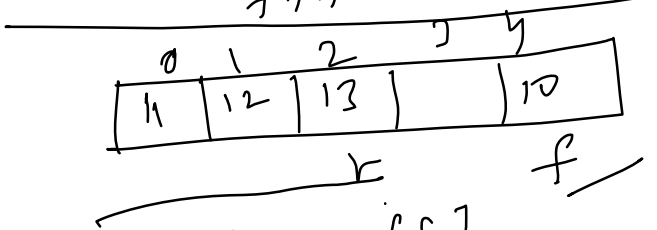
```
void cqinsert(int x)
{
    if((f==0 && r==size-1) || (f==r+1))
    {
        printf("\nQueue overflow");
        return;
    }
    if(r==size-1)
        r=0;
    else
        r++;
    cq[r]=x;
    if(f==-1)
        f=0;
}
```

Deletion



q = cq[f]

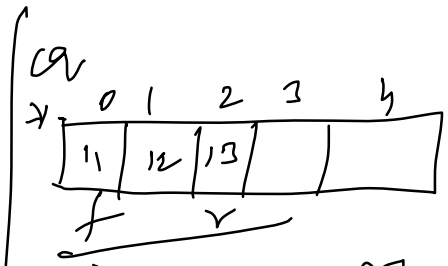
f++



10 = cq[f]

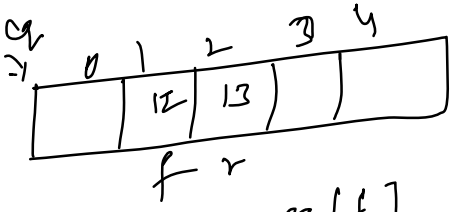
∵ f == size-1

f = 0



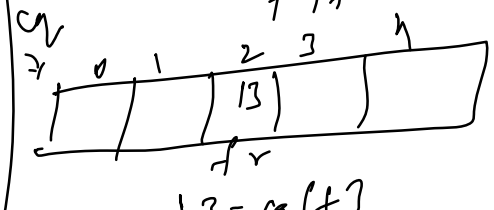
11 = cq[f]

f++



12 = cq[f]

f++



13 = cq[f]

∵ f == r

f = r = -1

```
int cqdelete()
{ int x;
  if(f==-1)
  {
  printf("Queue underflow");
      return -1;
  }
  x = cq[f];
  if(f==size-1)
     f=0;
  else if(f==r)
     f=r=-1;
  else
     f++;
  return x;
}
```

Display :-

```
void display()
{
  int i;
  if(f<=r)
  {
    for(i=f; i<=r; i++)
    {
      printf("%d ", cq[i]);
    }
  }
  else
  {
    for(i=f; i<=size-1; i++)
    {
      printf("%d ", cq[i]);
    }
    for(i=0; i<=r; i++)
    {
      printf("%d ", cq[i]);
    }
  }
}
```

O(size)

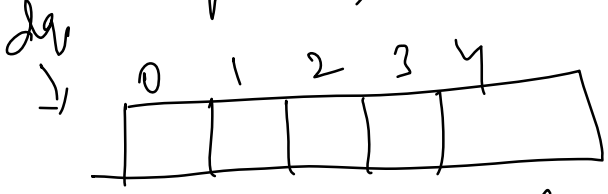
Double Ended Queue :- A dqueue is a queue where insertions are allowed from front and rear and deletions are allowed from front and rear positions.

Operations

1) Insertion from rear
2) Insertion from front
3) Deletion from rear
4) Deletion from front
5) Display
6) Front Element
7) Rear Element
8) Size/Length/count
9) isfull
10) isempty

Insertion from rear

dq[size], size = 5

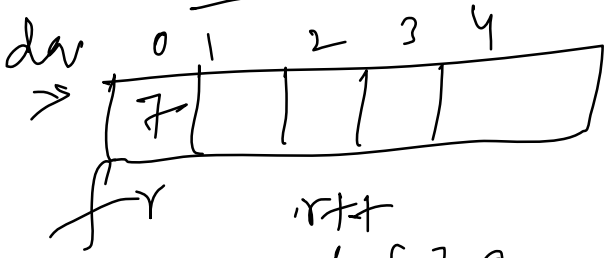


f = r = -1

rear++

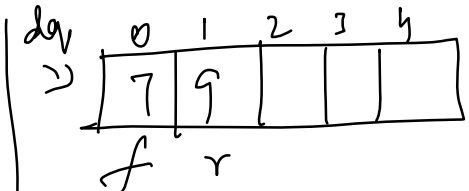
if (f == -1) f = 0

dq[r] = 7



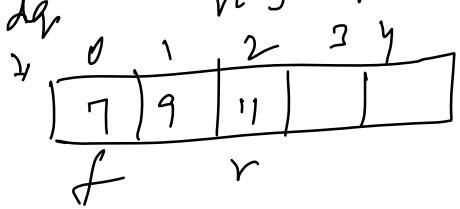
rear++

dq[r] = 9



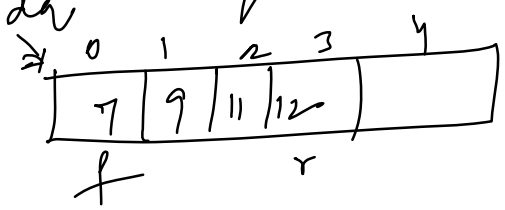
rear++

dq[r] = 11



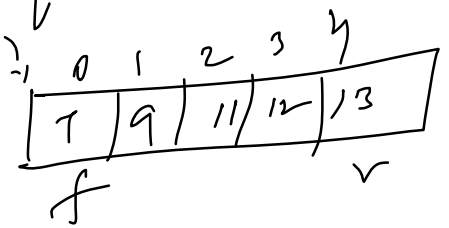
rear++

dq[r] = 12



rear++

dq[r] = 13

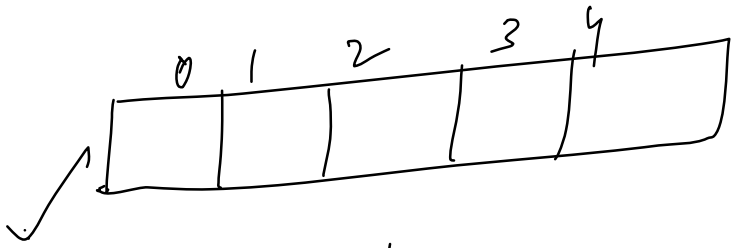


∴ r == size - 1

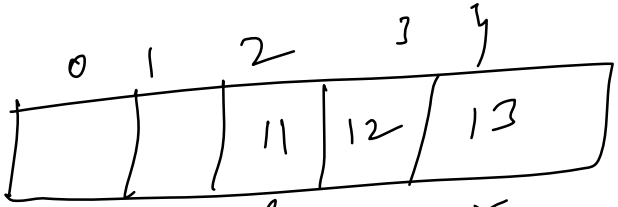
dqueue overflow

```
void dqinsertrear(int x)
{
  if(r==size-1)
  {printf("dqueue overflow");
    return;
  }
  r++;
  dq[r]=x;
  if(f==-1)
    f=0;
}
```

Insertion from front

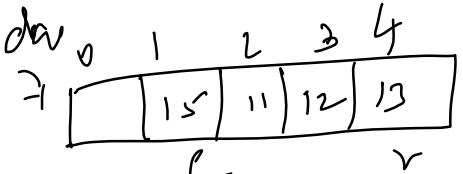


r=f=-1



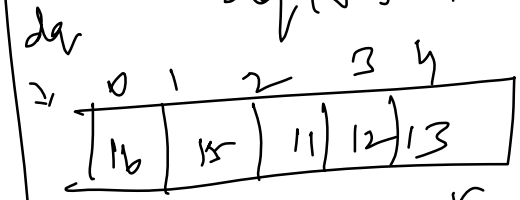
f--

dq[f]=15



f--

dq[f]=16

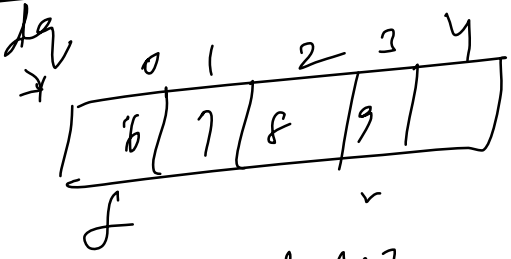


∴ f==0

overflow

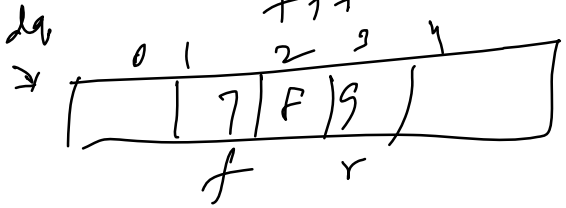
```
void dq_insert_front(int x)
{
  if (f==0)
  {
  printf("Queue Overflow");
    return;
  }
  if(f==-1)
    f=r=0;
  else
    f--;
    dq[f]=x;
}
```

Delete from front



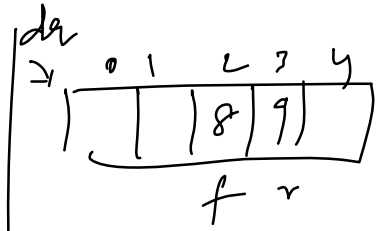
x = dq[f]

f++



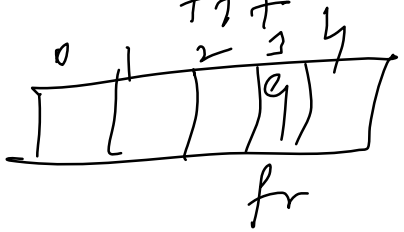
x = dq[f]

f++



x = dq[f]

f++



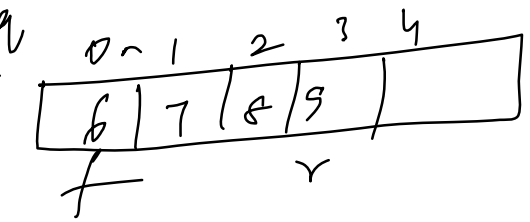
x = dq[f]

if f == r

f = r = -1

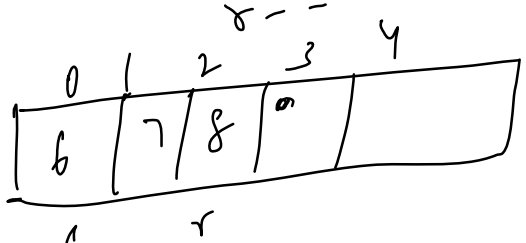
```
void dqdeletefront()
{
int x;
if (f == -1)
{
printf("Queue Underflow");
return -1;
}
x = dq[f];
if (f == r)
f = r = -1;
else
f++;
return x;
}
```

Delete from rear



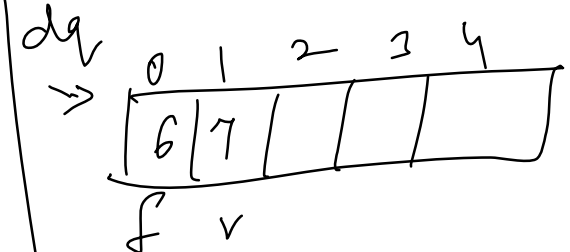
$x = dq[r]$

$r--$



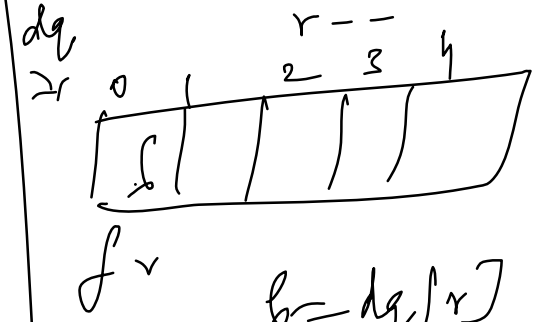
$8 = dq[r]$

$r--$



$7 = dq[r]$

$r--$



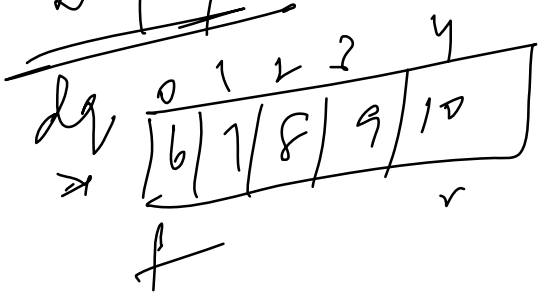
$6 = dq[r]$

$\therefore f == r$

$f = r = -1$

```
int dqdeleterear()
{ int x;
  if (r == -1)
  {
  printf("dequeue empty\n");
    return -1;
  }
  x = dq[r];
  if (f==r)
    f=r=-1
  else
    r--;
  return x;
}
```

Display




```
void dqdisplay()
{ int i;
  if (f==-1)
  { printf("Dqueue empty\n");
    return;
  }
  for(i=f; i<=r; i++)
  { printf("%d ", dq[i]);
  }
}
```

Applications of Stack

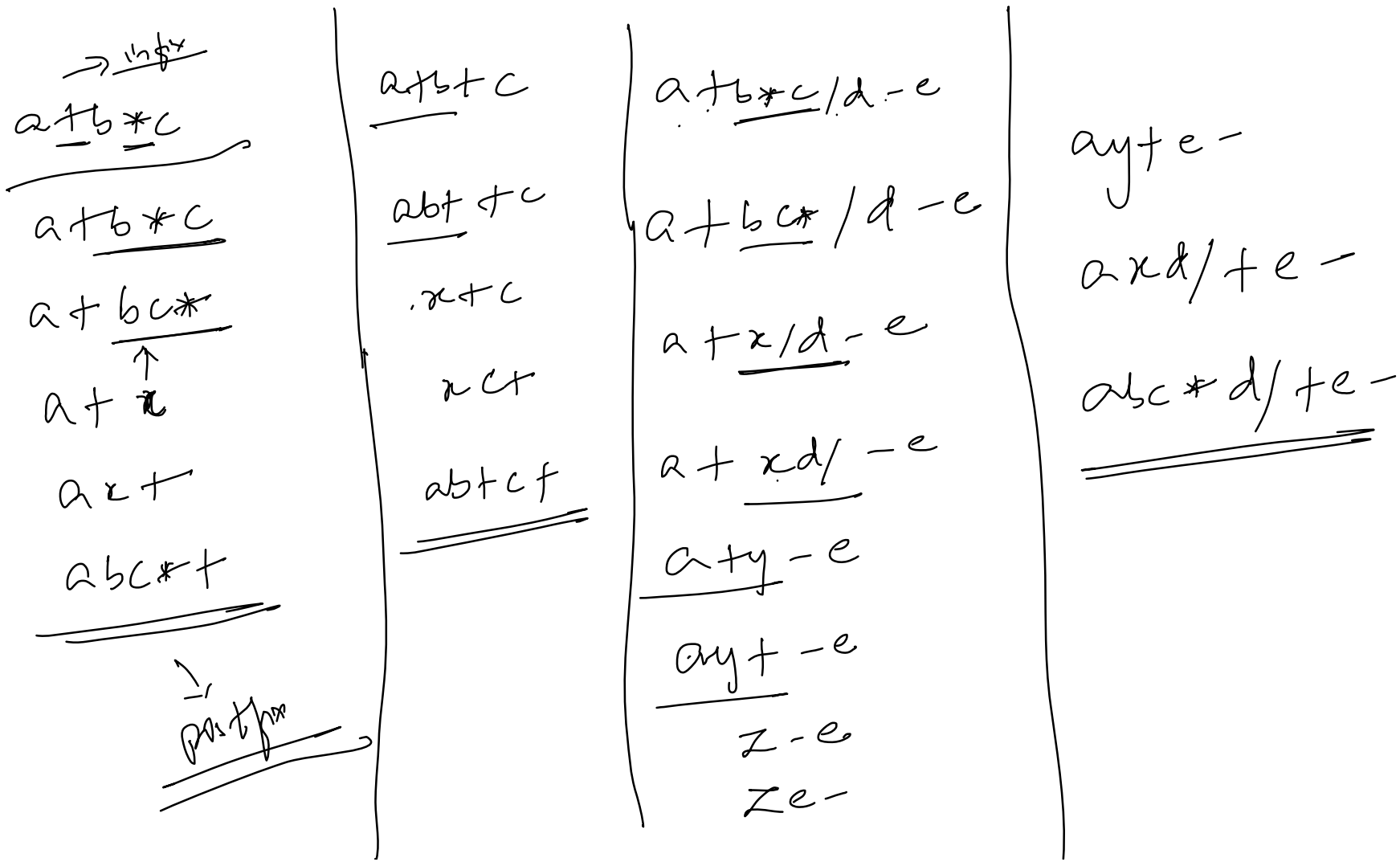
1) Conversion of expressions
2) Evaluation of a postfix expression
3) Checking whether the parentheris are valid or not
4) checking whether a string is palindrome or not
5) Recursion

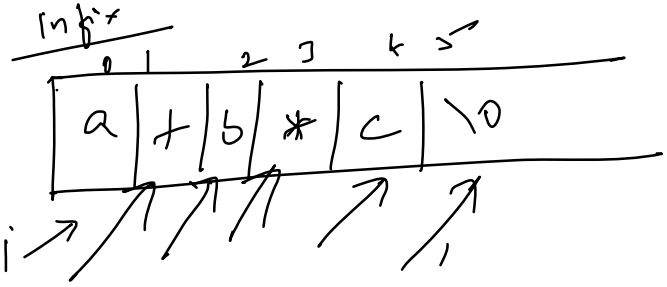
Expressions are of 3 types

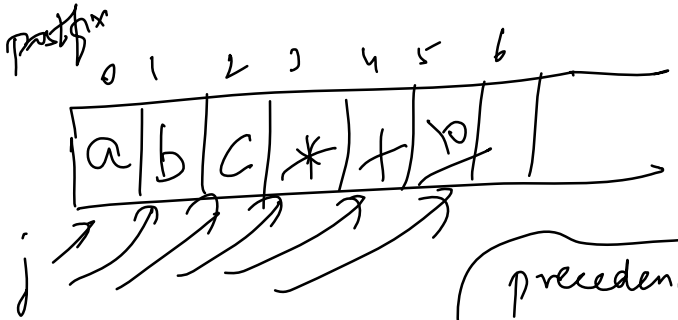
infix expression — Eg:- a+b

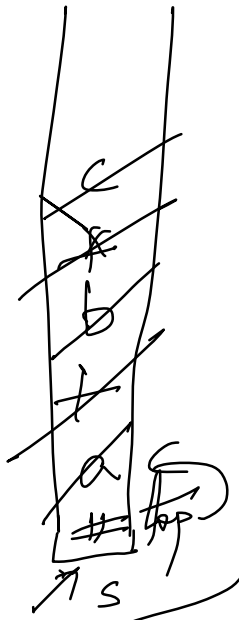
Prefix expression — +ab

postfix expression — ab+


a+b*c
a+b*c
a+bc*
a+x
ax+
abc*+
postfix
ab+c
ab+c
xc+
ab+cf
a+b*c/d-e
a+bc*/d-e
a+x/d-e
a+xd/-e
a+y-e
ay+-e
z-e
ze-
ay+e-
axd/+e-
abc*d/+e-



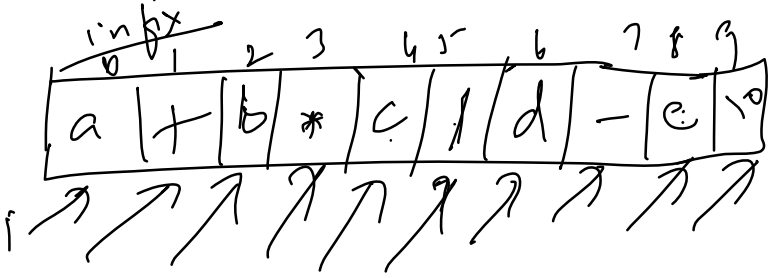




character/operator	precedence
#	0
+, −	1
*, /, %	2
a, b, c, -----	3

push :- if infix expression's character's precedence > stack's top most elements precedence

pop :- if infix expression's character's precedence <= stack's top most element, pop the character from stack and add it to postfix





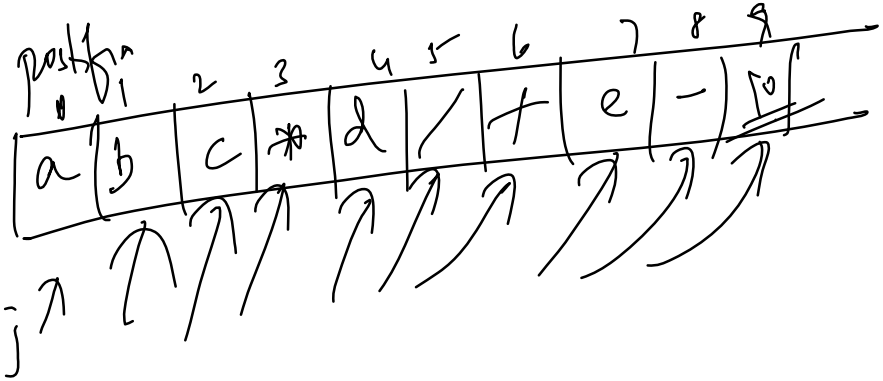
precedence / priority

\# → 0

+, − → 1

*, /, % → 2

a, 1, c, ---- → 3



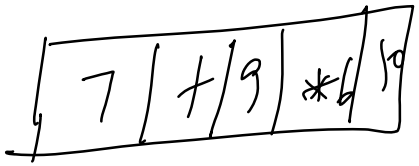
```
void push(char ch)
{
    top++;
    s[top]=ch;
}

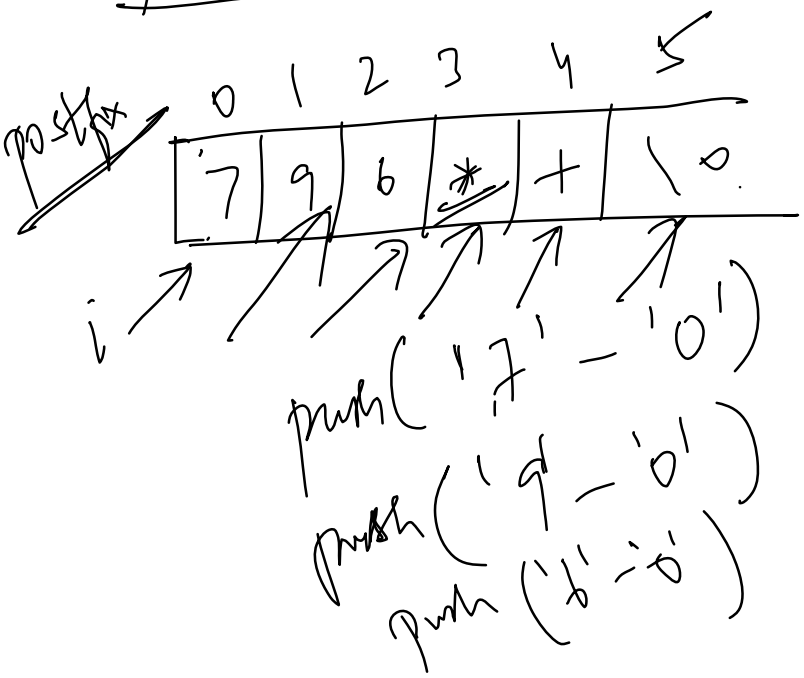
char pop()
{
    char ch;
    ch= s[top];
    top--;
    return ch;
}

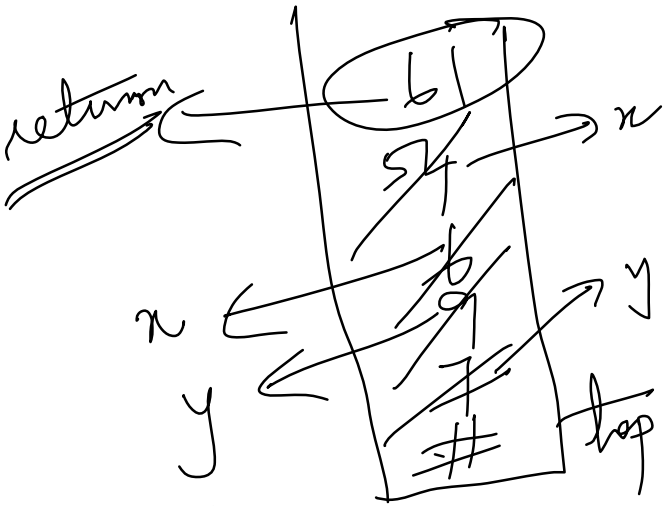
int priority(char ch)
{
    switch(ch)
    {
    case '+':
    case '-': return 1;
    case '*':
    case '/':
    case '%': return 2;
    case '#': return 0;
    default: return 3;
    }
}
```

```
void convert(char infix[], char postfix[])
{
    int i=0, j=0;
    while(infix[i] != '\0')
    {
        while( priority(infix[i]) <= priority(s[top]))
        {
            postfix[j] = pop();
            j++;
        }
        push(infix[i]);
        i++;
    }
    while(s[top] != '#')
    {
        postfix[j] = pop();
        j++;
    }
    postfix[j] = '\0';
}
```

Evaluation of a postfix expression







push(y * x)

push(y + x)

0-48
1-49
2-50
3-51
4-52
5-53
6-54
7-55
8-56
9-57

Steps

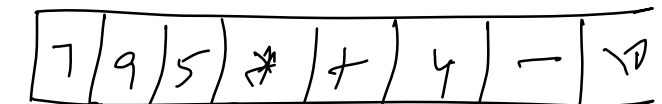
1) If it is a value (0-9) push the value into the stack
2) If it is an operator pop 2 values from the stack, apply the operator on the popped values, and then push the output value of operation into stack
3) Repeat 1 & 2 until '\0' is encountered in postfix expression.
4) Return the top most value of stack by popping the value.

7+9*5-4

7+95*-4

795*+-4

795*+4←



push('7'-'0')

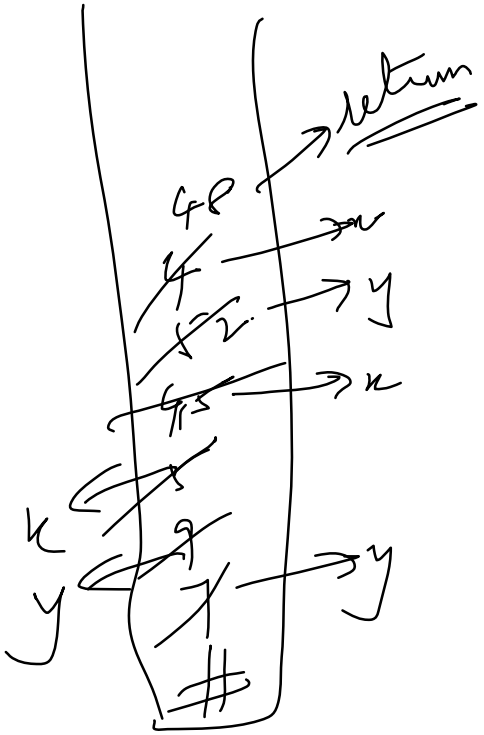
push('9'-'0')

push('5'-'0')

push(y*x)

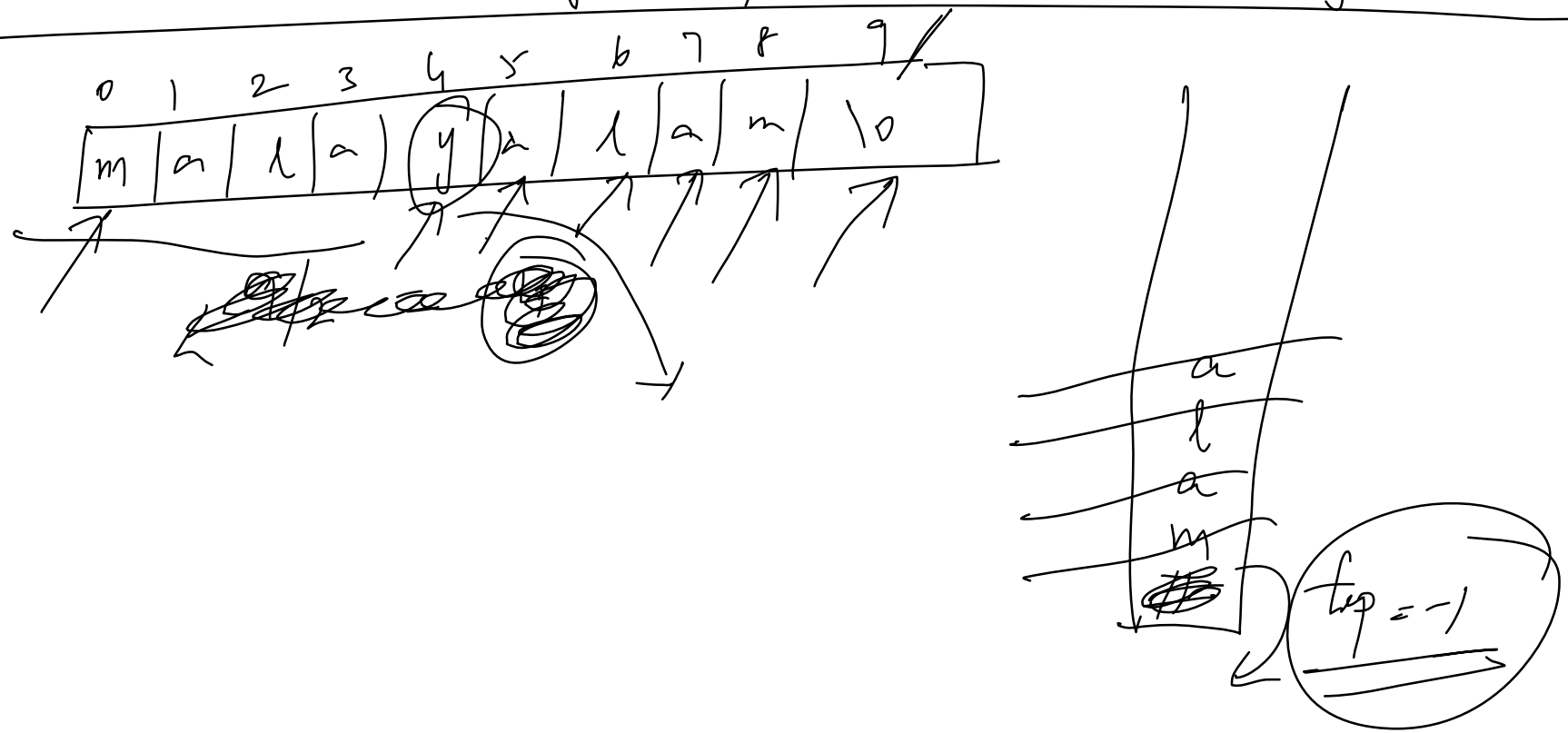
push(y+x)

push(y-x)



```
int evaluate(char postfix[])
{
    int i, x, y;
    for(i=0; postfix[i] != '\0'; i++)
    {
        if( postfix[i] >= '0' && postfix[i] <= '9')
            push( postfix[i] - '0')
        else
        {
            x = pop();
            y = pop();
            switch( postfix[i])
            {
                case '+': push(y+x);
                          break;
                case '-': push(y-x);
                          break;
                case '*': push(y*x);
                          break;
                case '/': push(y/x);
                          break;
                case '%': push(y%x);
                          break;
            }
        }
    }
    return pop();
}
```

Check whether a string is palindrome or not using stack



1) Find the length of the string
2) Move characters into stack till half of the length - 1
3) If it is an even length string skip next character
4) For the remaining characters pop & compare. If atleast 1 character is not equal to popped value it is not a palindrome. Else it is a palindrome.

Recursion

$5! = 5 \times 4 \times 3 \times 2 \times 1$

(120)

$fact(5) = 5 \times 4 \times 3 \times 2 \times 1$ (24)

```
unsigned long int fact(int n)
{
  if (n==0)
  return 1;
  else
  return n*fact(n-1);
}
```

return $5 \times fact(4)$

return $4 \times fact(3)$

return $3 \times fact(2)$

return $2 \times fact(1)$

return $1 \times fact(0)$

return 1

(1*1)

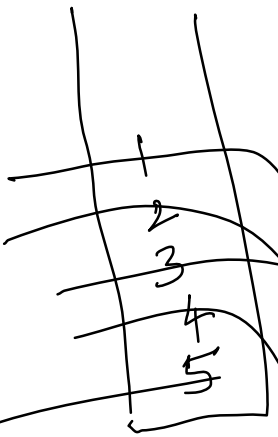
$5! = 5*4!$

return $4*3!$

return $3*2!$

return $2*1!$

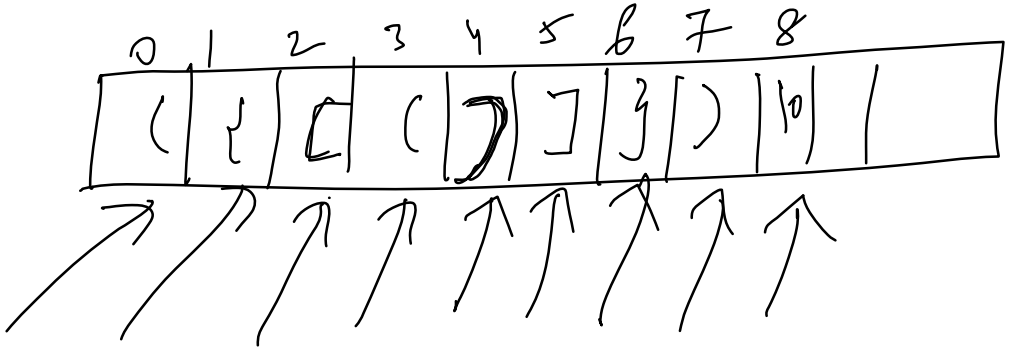
return $1*0!$



Checking whether given parenthesis / braces / brackets are matching or not.

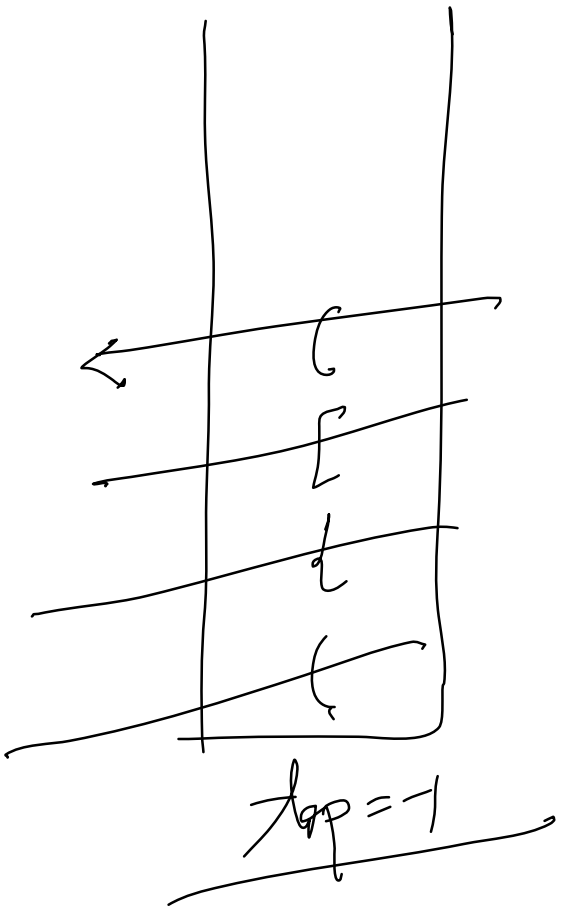
( ) , ([ ]) , ({ [( )] }) ({ [(})])

({ } ( ) [ ] { [( )] })



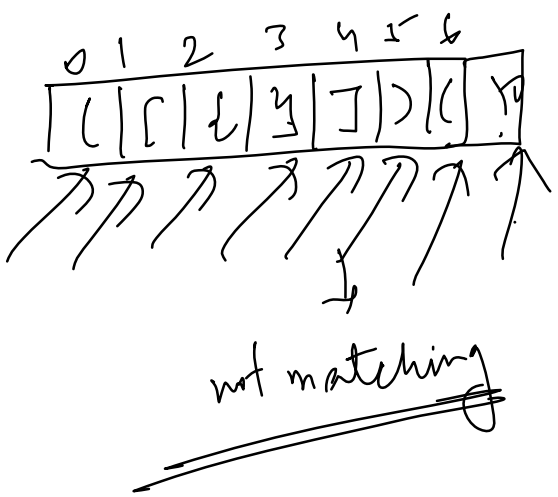
1) If we encounter opening (/{/[we push it into the stack.

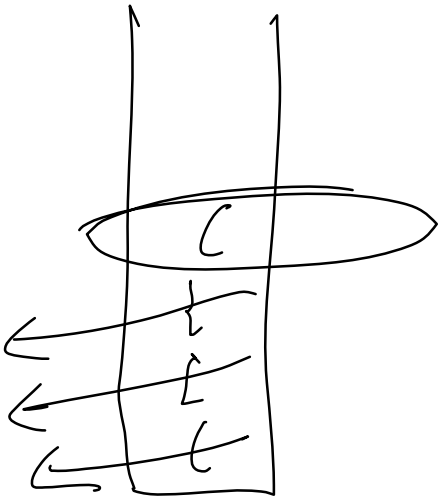
2) If closing)/}/] is encountered a value is popped from stack and checked whether it is opposite counterpart of the value encountered in array and if it is opposite counter part we move to next

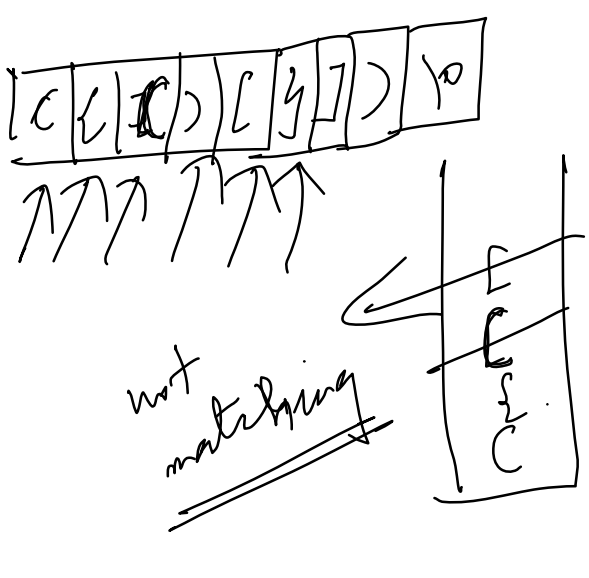


character and repeat step 1 & 2 until '\0' is encountered in string.

3) If stack is empty after steps 1 & 2 we say parenthesis are matching else parenthesis are not matching.



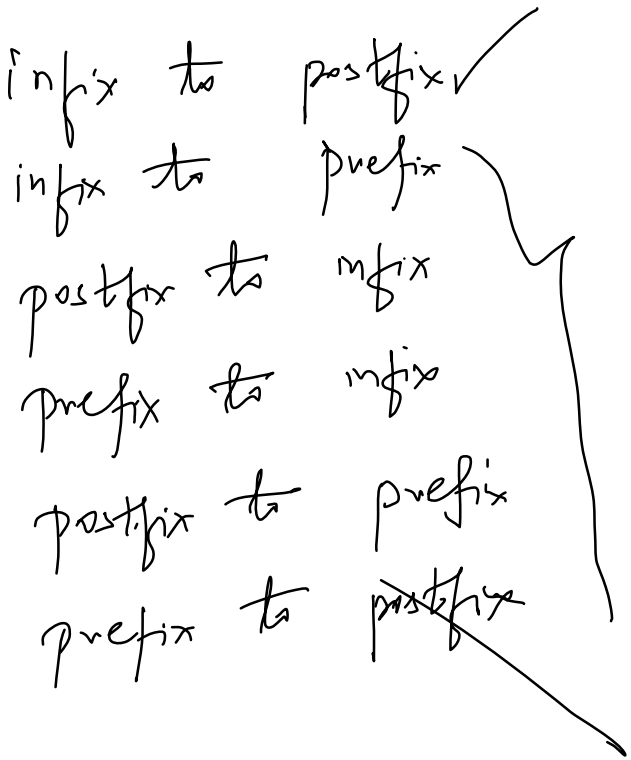




```
int matching(char s[])
{
    int i; char temp;
for(i=0; s1[i]!='\0'; i++)
{
    if(s1[i]=='('||s1[i]=='{'||s1[i]=='[')
        push(s1[i])
    else
    {
        if(s1[i]==')')
        {
            temp = pop();
            if(temp!='(')
                return 0;
        }
        if(s1[i]=='}')
        {
            temp = pop();
            if(temp!='{')
                return 0;
        }
    }
```

```
if(s1[i] == ']')
{
    temp = pop();
    if(temp != '[')
        return 0;
}
}
}
```

```
if(top == -1)
    return 1;
else
    return 0;
}
```


infix to postfix ✓
infix to prefix
postfix to infix
prefix to infix
postfix to prefix
prefix to postfix

Conversion of Expressions

1) Infix to Postfix

a) get a character from infix

b) if its priority is greater than stack's top most element priority push it into the stack and go to the next char of infix.

c) if its priority is <= stacks top most element priority pop and place it in postfix expression and continue the comparison till infix elements priority is > stacks top most element priority

d) repeat steps b. and c until '\0' is encountered in infix

e) copy the remaining values of stack by popping into postfix expression and add at '\0' to convert it to string.

Infix to Prefix

1) Reverse the infix string
2) Apply infix to postfix logic on reversed string
3) Reverse the output string to get prefix string

a+b*c

a+b*c

a+x

+ax

+a*b*c

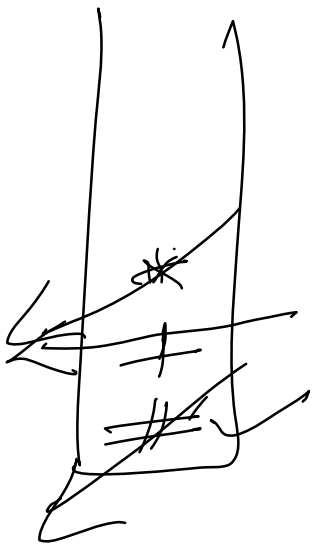
prefix to infix

prefix

+	a	*	b	c	\0

infix

a	+	b	*	c	\0



1) if character is an operator push it into stack
2) if it is not an operator add it to infix, pop a value from stack, check if it is an operator and add it to infix
3) Repeat 1 & 2 until null character is encountered.

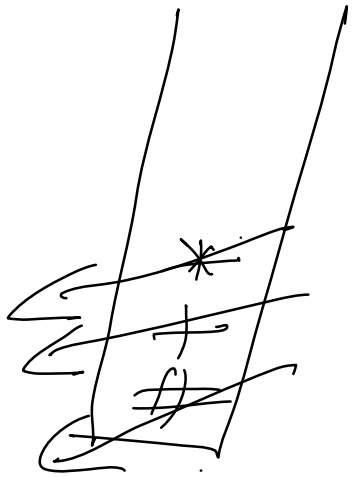
postfix to infix

postfix

a	b	c	*	+	\0

postfix

+	*	c	b	a	\0



infix

c	*	b	+	a	\0

infix

a	+	b	*	c	\0

1) reverse postfix expression
2) apply prefix to infix
3) reverse the output expression to get equivalent prefix expression.

postfix to prefix

1) postfix to infix
2) infix to prefix

prefix to postfix

prefix to infix
infix to postfix

Linked List :- A linear data structure where the data is present in continuous memory locations logically, but may or may not be present in continuous memory locations physically.

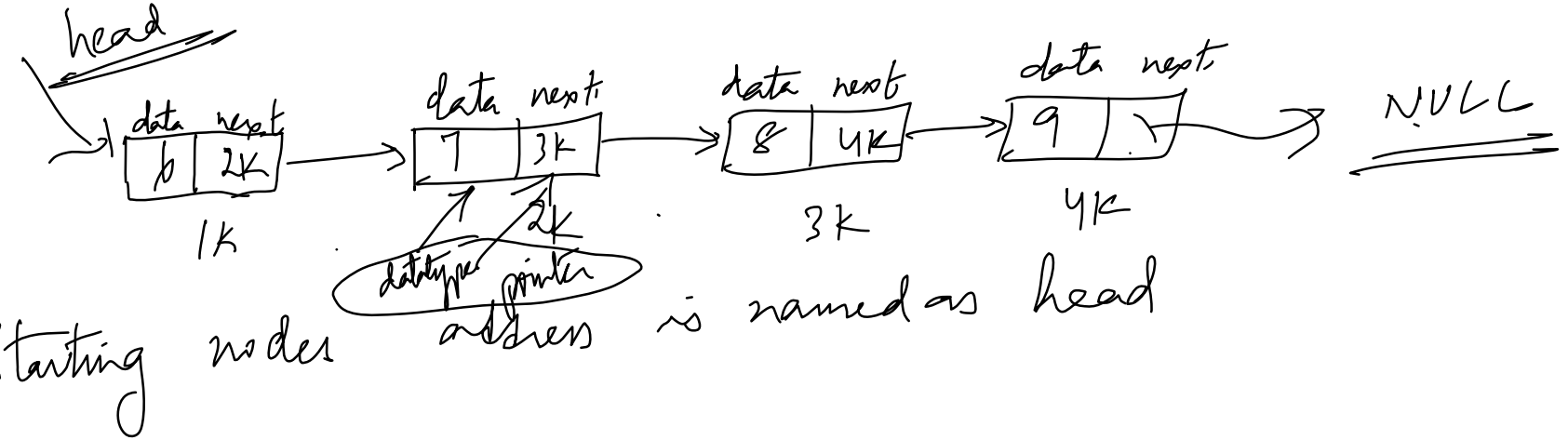
Linked list is a collection of nodes (data + address) which are allocated memory dynamically (using malloc) whenever required and can be added to the list whenever required.

Types of linked lists

1) Single linked list
2) Circular linked list
3) Double linked list
4) Doubly Circular linked list.

Single linked list

A linked list where every node maintains the address of the next node and the last node's next is NULL.



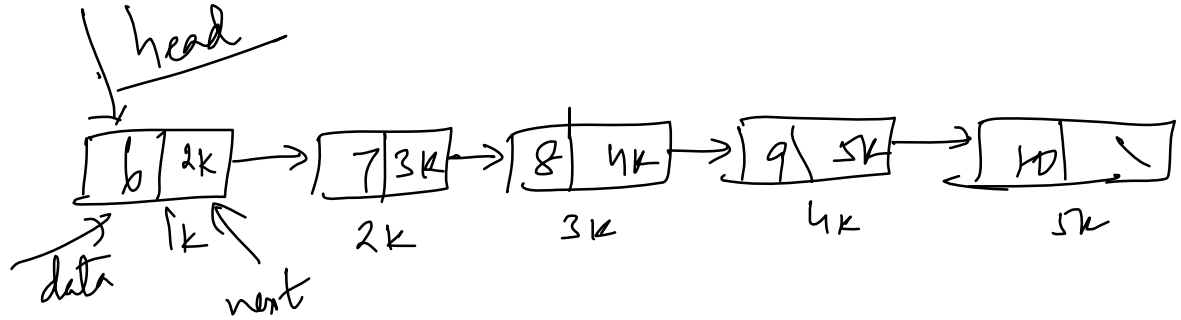
Starting node address is named as head

Every node is a collection of data (int / char / float / double etc / combinations) and address (pointer of the same type). Hence we define node using a structure as follows.

```
struct node
{
    datatype member1;
    datatype member2;
    :
    struct node *next;
};
```

Self Referential Structure

Eg:-



definition of node of the above example

```
struct node
{
    int data;
    struct node *next;
} *head = NULL;
```

→ initially

Operations

1) create / append / addatend
2) display
3) addatbeg
4) addataposition
5) delete by value
6) delete by position
7) search
8) reverse.

Allocation of memory dynamically to a node

To allocate memory dynamically we use malloc()

void* malloc(int no:of bytes)

```
struct node
{
  int data;
  struct node *next;
}*head = NULL;
```

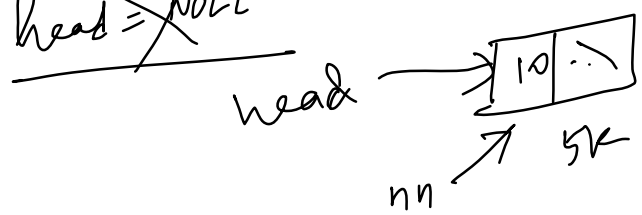
```
struct node *nn;
nn = (struct node*) malloc(sizeof(struct node));
nn -> data =
nn -> next =
```

Create :-

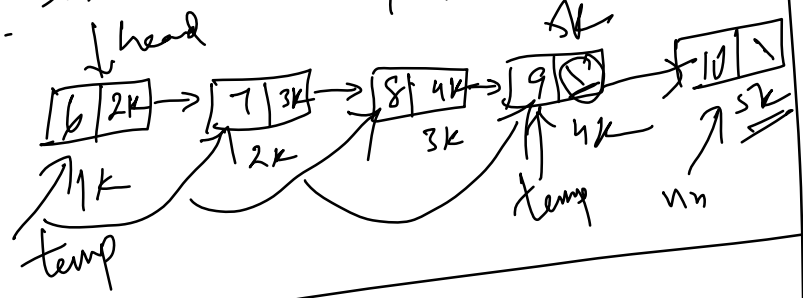
1) create a node (allocate memory dynamically)
2) If list is not present → the newly created node becomes the list (we make head point to that node)
3) If list is already present → we move to the last node of the list and make the newly created node as the new last node.

list is not present

head = ~~NULL~~



list is already present

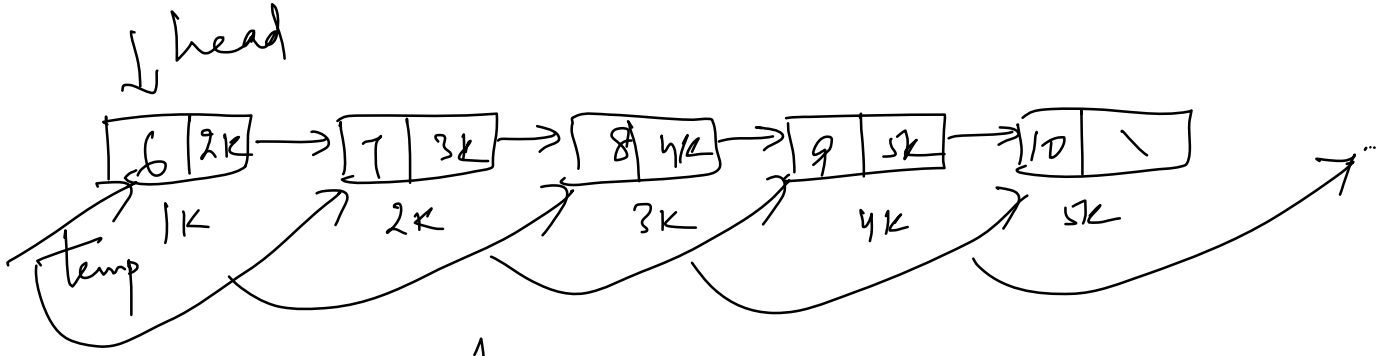



```
struct node
{
    int data;
    struct node *next;
} *head = NULL;
```

```
void create(int x)
{
    struct node *nn, *temp = head;
    nn = (struct node*) malloc( sizeof(struct node));
    nn -> data = x;
    nn -> next = NULL;
// list is not present
    if (head == NULL)
    {
        head = nn;
        return;
    }
// list is already present
    while (temp -> next != NULL)
    {
        temp = temp -> next;
    }
    temp -> next = nn;
}
```

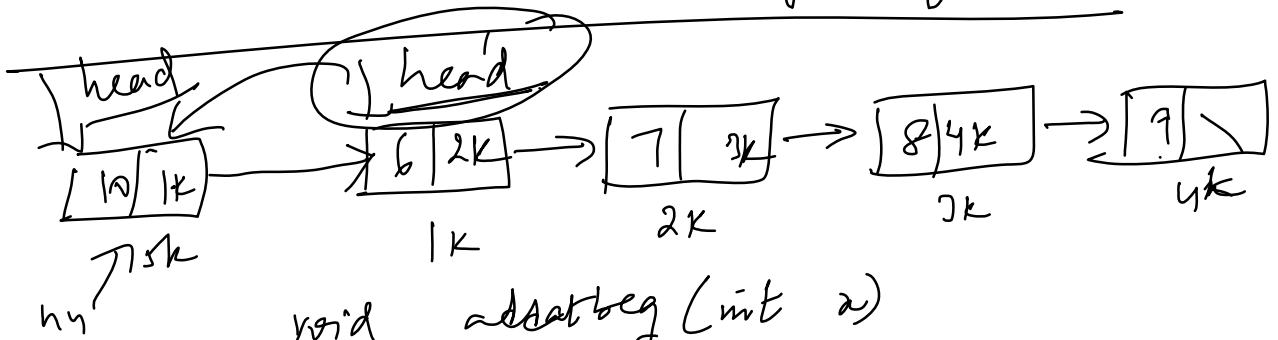
6 7 8 9 10

Display




```
void display()
{
    struct node *temp=head;
    if(head==NULL)
    {
        printf("List is empty");
        return;
    }
    while(temp!=NULL)
    {
        printf("%d ", temp->data);
        temp=temp->next;
    }
}
```

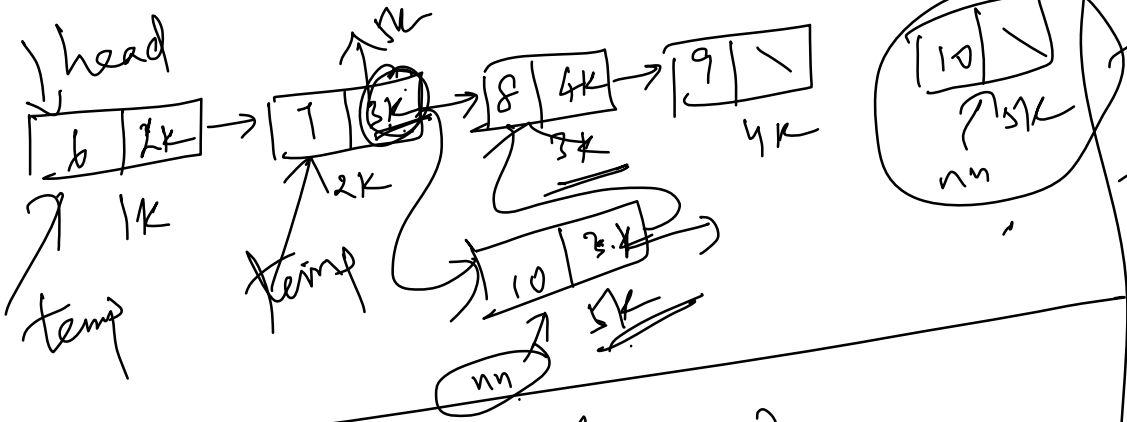
Add a node at the beginning



1) Create node
2) store data
3) update next of new node with head
4) make head point to nn

```
void addatbeg(int x)
{
  struct node *nn;
  nn=(struct node*) malloc(sizeof(struct node));
  nn->data = x;
  nn->next = head;
  head= nn;
}
```

Add at a Specified Position (p)




```
for(i=1; i< p-1; i++)
{
    if(temp->next == NULL)
        break;
    temp = temp->next;
}
```

```
if (p<=1)
    addatbeg(x);
```

To add at node at the nth position of the list we must come to $(n-1)^{th}$ position of the list

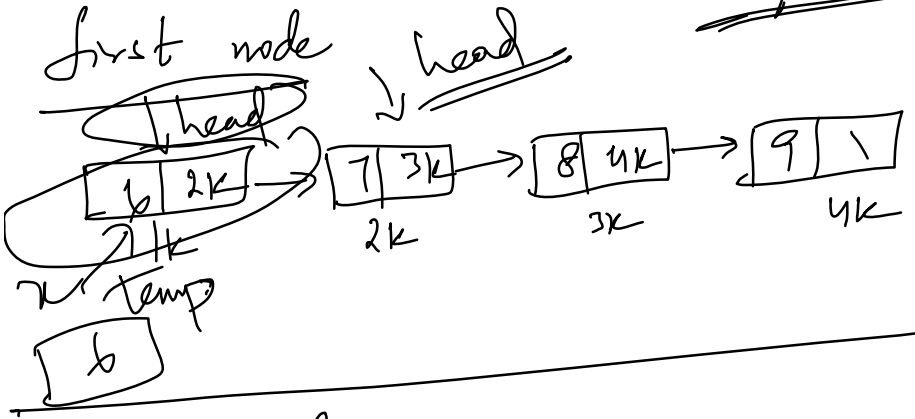
```
nn->next = temp->next;
temp->next = nn;
```

```
void addatpos( int x, int p)
{
  struct node *nn, *temp=head;
    if( p<=1)
    {
      addatbeg(x);
      return;
    }
    for(i=1; i<p-1; i++)
    {
      if( temp->next==NULL)
        break;
      temp= temp->next;
    }
    nn=(struct node*)malloc(sizeof(struct node));
      nn->data=x;
    nn->next= temp->next;
    temp->next=nn;
}
```

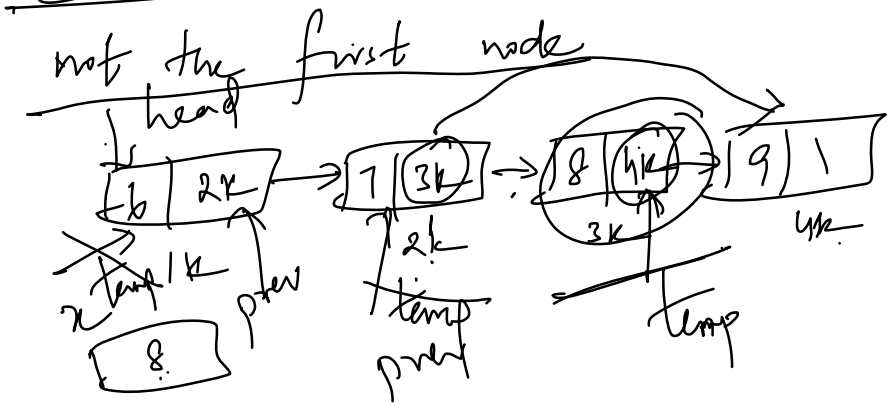
Delete by Value

temp = head

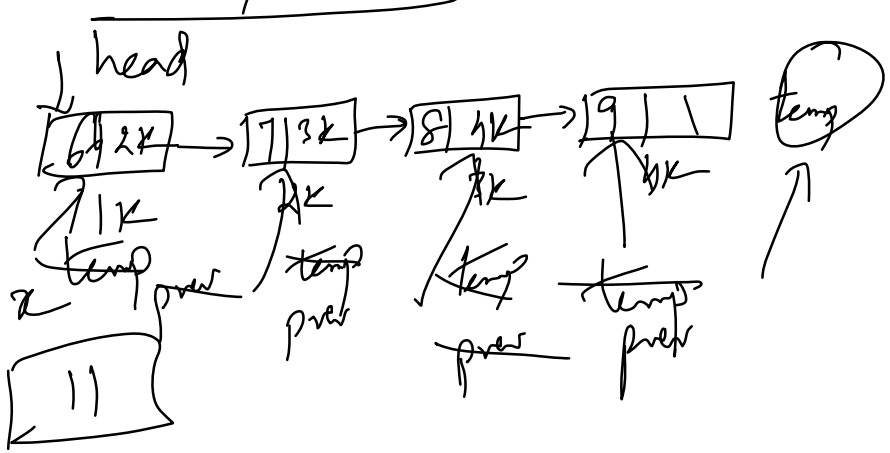
first node



not the first node

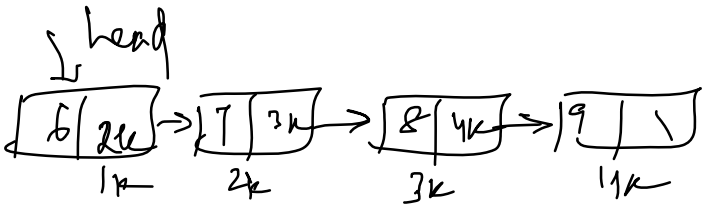


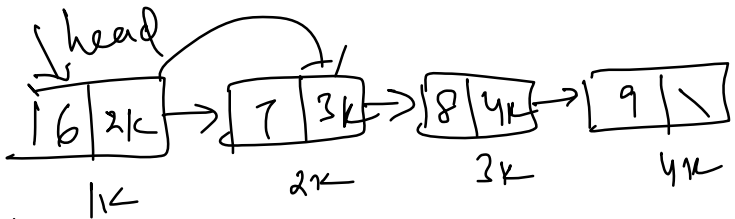
not present

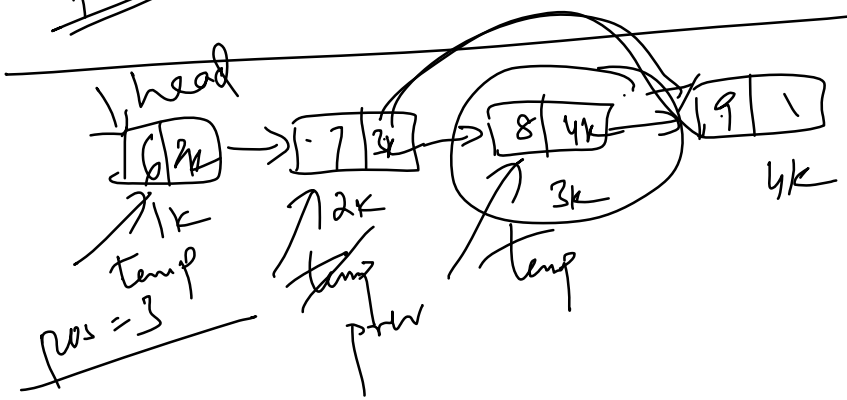


```
void ldelete(int x)
{
  struct node *temp=head, *prev;
  while(temp!=NULL)
  {
    if(x==temp->data)
    {
      if(temp==head)
      {
        head=temp->next;
      }
      else
      {
        prev->next=temp->next;
      }
      printf("Node deleted");
      free(temp);
      return;
    }
    prev=temp;
    temp=temp->next;
  }
  printf("Element not found");
}
```

Delete by position :-





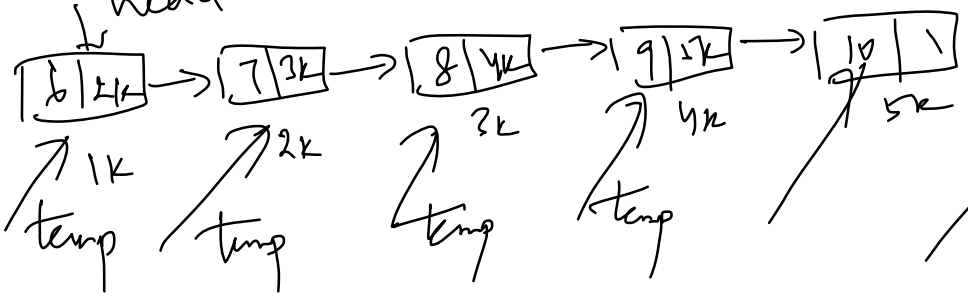


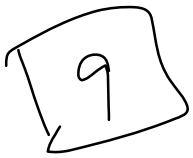

```
if(p==1)
{
    head = temp->next;
    free(temp);
    return;
}
```

printf("

```
for( i=1; i<p; i++)
{
    if(temp == NULL)
    {
        printf("Pos not found");
        return;
    }
    prev = temp;
    temp = temp->next;
}
prev->next = temp->next;
printf("Node deleted");
free(temp);
}
```

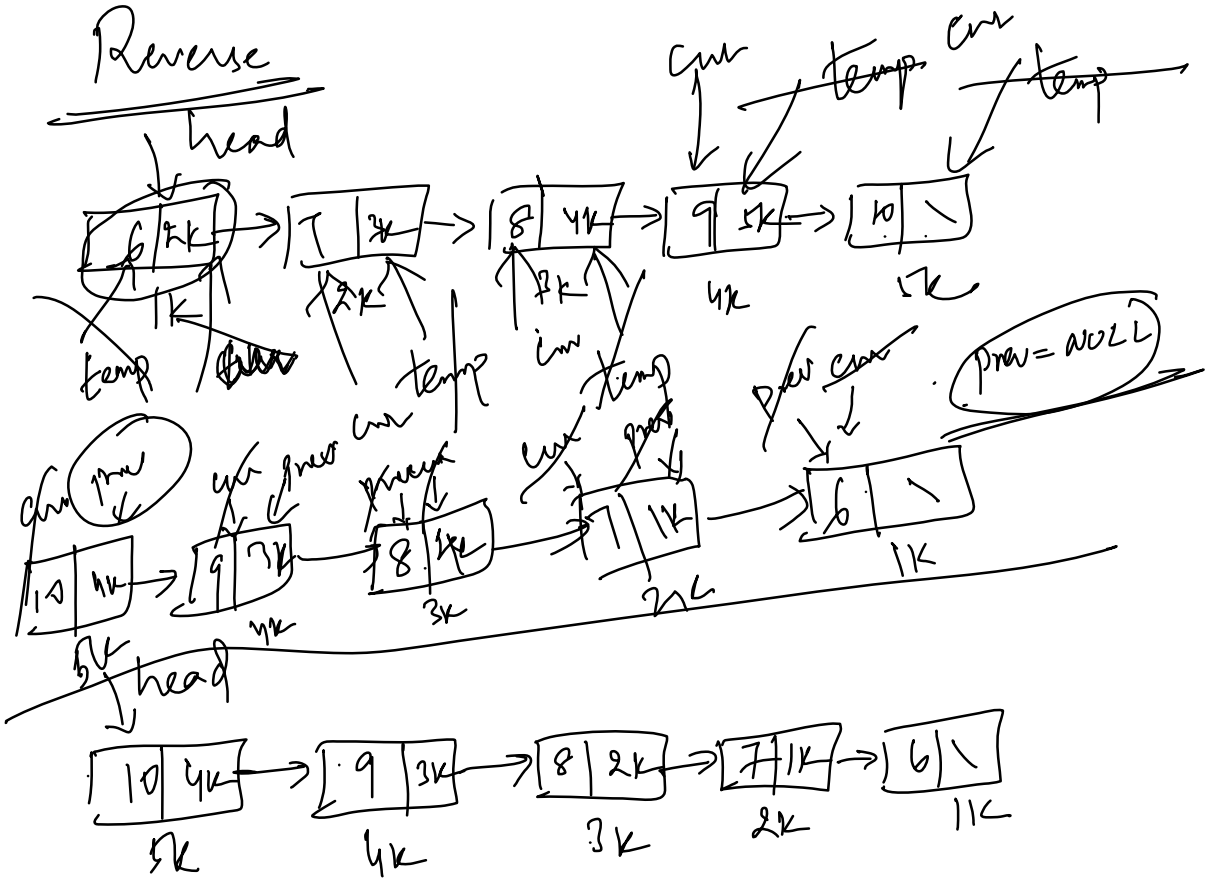
Search :-



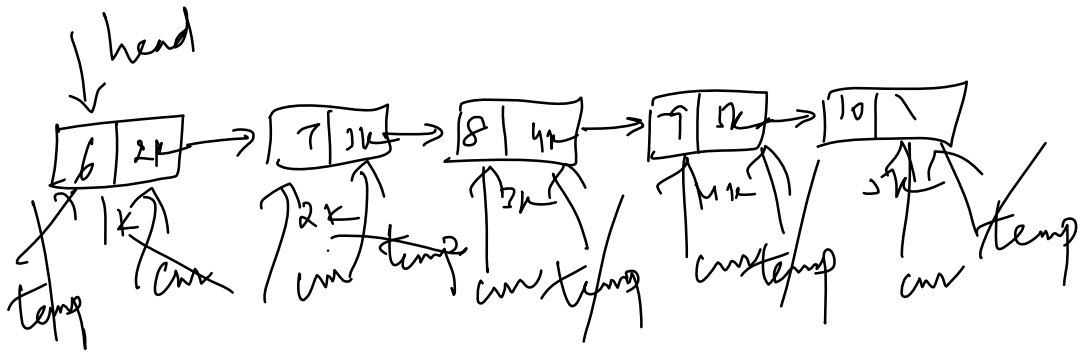





```
while (temp != NULL)
{
    if (temp->data == a)
    {
        printf ("Element found\n");
        return;
    }
    temp = temp->next;
}
printf ("Node not found");
```

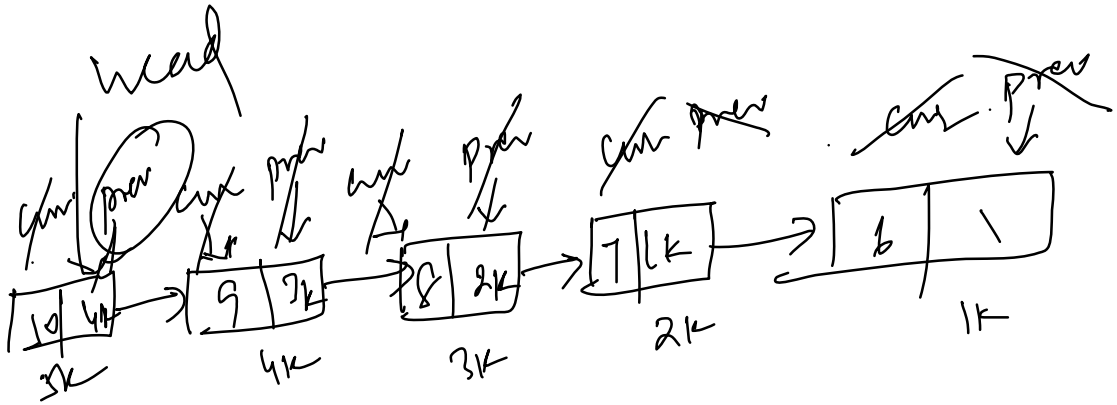



```
while(temp != NULL)
{
    temp = temp->next;
    cur->next = prev;
    prev = cur
    cur = temp;
}
head = prev;
```



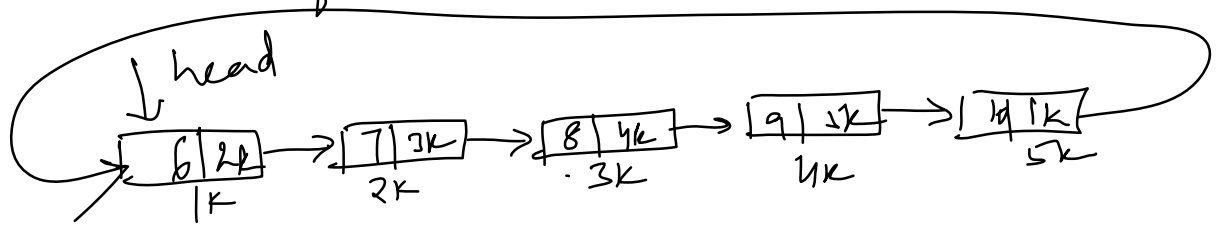



```
prev=NULL
while (temp!=NULL)
{
  temp=temp->next;
  cur->next=prev;
  prev=cur
  cur=temp
}
head=prev;
```



Circular linked list

A circular linked list is a single linked list where the last node of the list stores the address of the first node



Operations

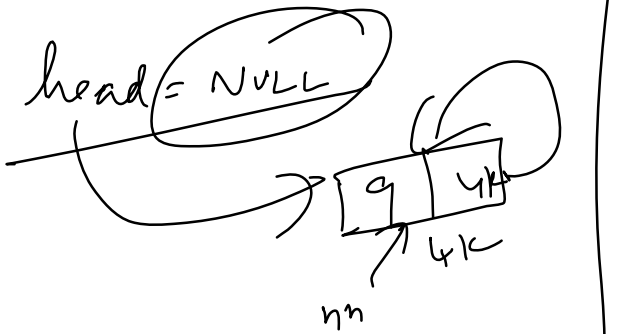
1) append / create
2) addat beg
3) addat a pos
4) deletebyvalue
5) display.

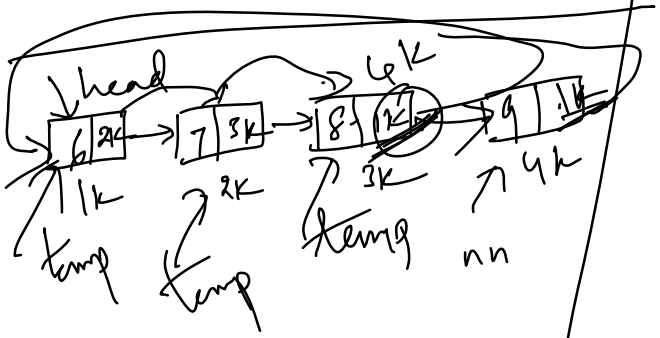
Node

```
struct node
{
    int data;
    struct node *next;
}*head = NULL;
```

Create :-

1) Create the node
2) if list is not present, the newly created node will me made to be pointed to by head.
3) If list is already present, this node is made the last node of the list and its next is made to points to head, the current last node's next will be pointing to newly created node.





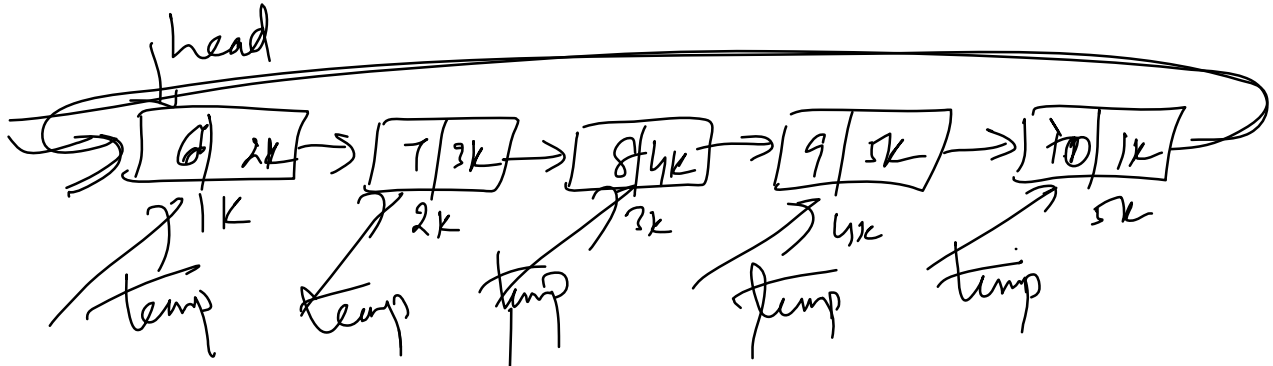

```
struct node *nn, *temp=head;
nn= (struct node*) malloc (sizeof(struct node));
nn-> data = x;

if ( head == NULL)
{
  head=nn;
  nn -> next=head;
  return;
}
```

```
while( temp->next != head)
{
  temp=temp->next;
}

temp->next = nn;
nn->next=head;
```

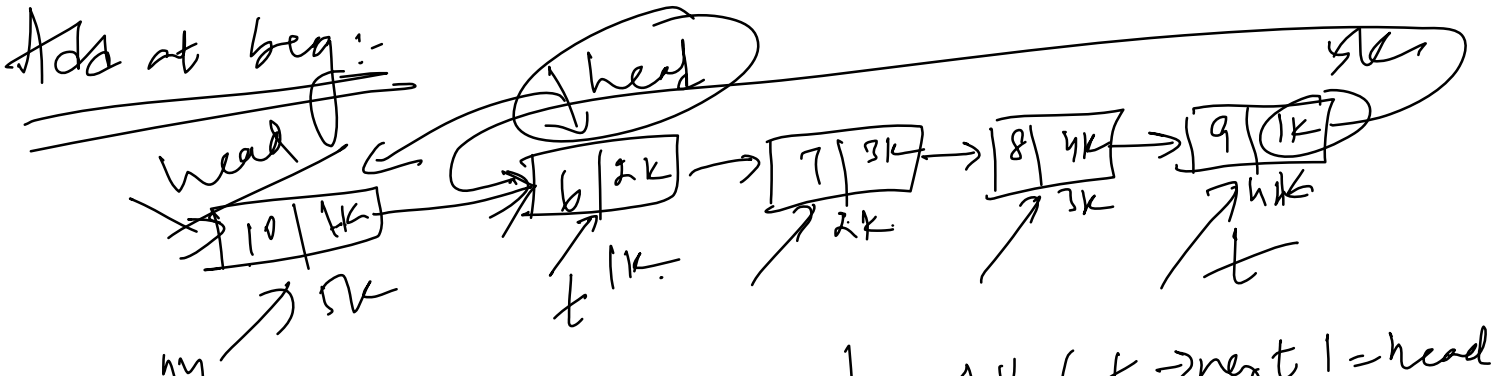
Display :-




```
struct node *temp = head;
if( head == NULL)
{
  printf("List is empty");
  return;
}
```

```
do
{
printf("%d ", temp -> data);
    temp = temp -> next;
} while(temp != head);
```

Add at beg :-




```
struct node *t = head;
nn = (struct node *)malloc(sizeof(struct node));
nn->data = x;
if (head == NULL)
{  head = nn;
   nn->next = head;
   return;
}
```

```
while(t->next != head)
   t = t->next;
{ nn->next = head; }
{ head = nn;
  t->next = head;
```

Add at a position

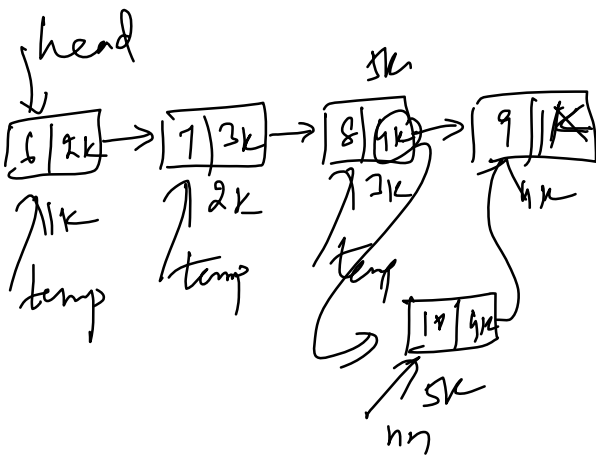
pos <= 1 → addatbeg

pos is not present → we will add the node at last

pos is present → add in the specified position

```
if(p<=1)
{
    addatbeg(x);
    return;
}

for(i=1;i<p-1;i++)
{
    if(temp->next==head)
        break;
    temp=temp->next;
}
nn=(struct node*)malloc
        (sizeof(struct node));
nn->data=x;
nn->next=temp->next;
temp->next=nn;
```



Deletion

if node to be deleted is headnode

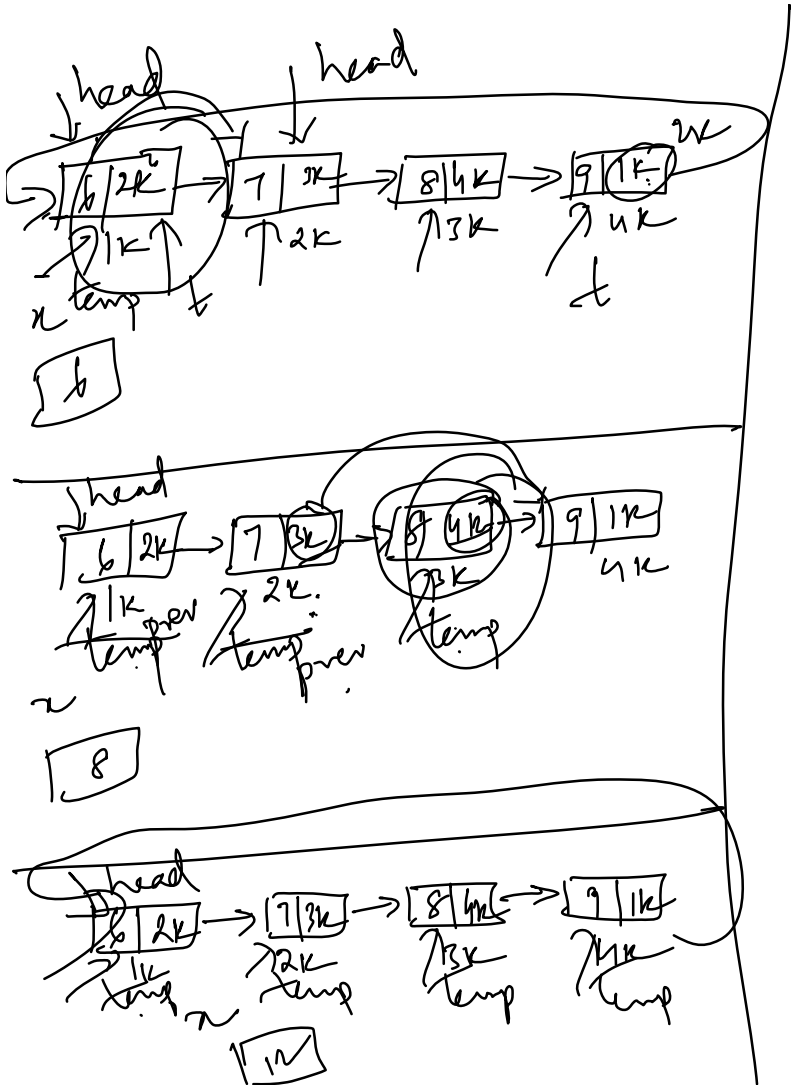
1) move to the last node of the list.
2) make head point to second node
3) make the last node point to the new head.

if node to be deleted is not head node

make prev node's next point to current node's (node to be deleted) next

if node to be deleted is not present

if in the process of searching we come back to headnode



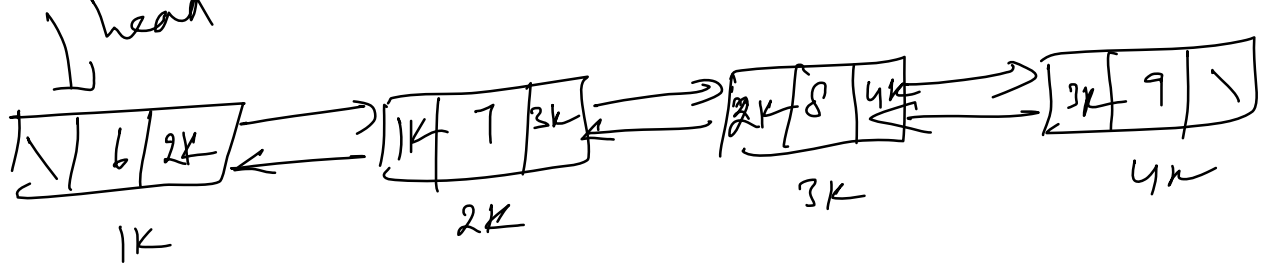

```
do
{
  if(temp->data == x)
  {
    if(temp == head)
    {
      {while(t->next != head)
        t=t->next;
      {head = temp->next;
      {t->next = head;
    }
```

```
else
{
 prev->next = temp->
                next;
}
 free(temp);
 printf("Deleted");
 return;
}  prev=temp;
 temp =temp->next;
 }while(temp!
        =head);

printf("\n'x' node
        not
        found");
```

Double Linked List:- A double linked list is a linked list where every node of the list maintains 2 addresses i.e, address of the previous node and address of the next node.

The previous address of the first node or head node is NULL and the next address of the last node is NULL.



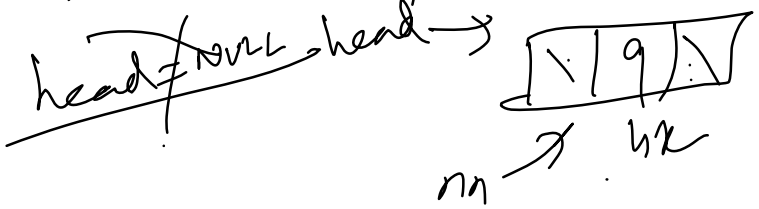
Operations

1) Create / append
2) display
3) addatbeg
4) addatpos
5) delete

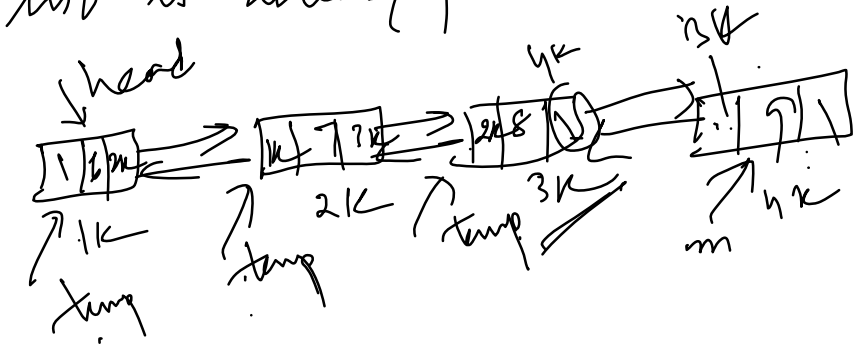
```
struct node
{
    int data;
    struct node *next;
    struct node *prev;
}*head=NULL;
```

Create :-

list is not present:

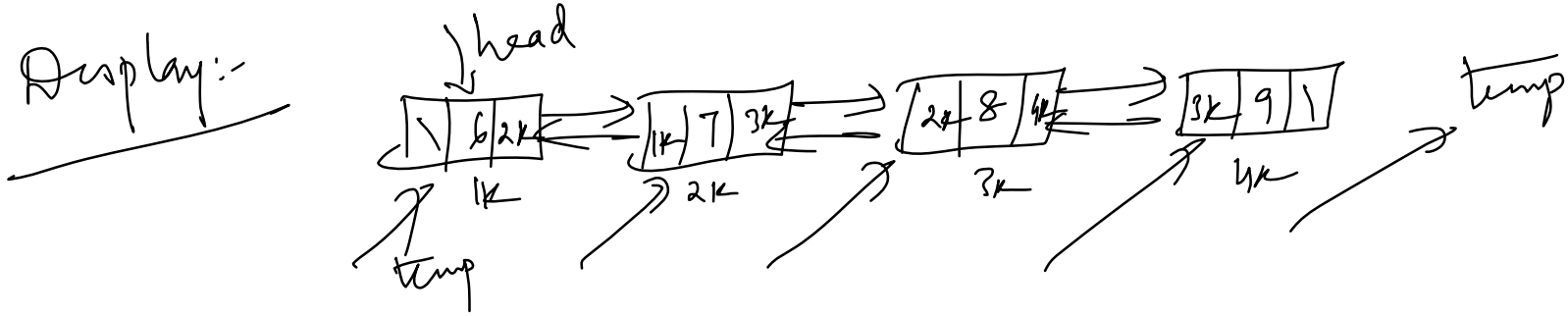


list is already present



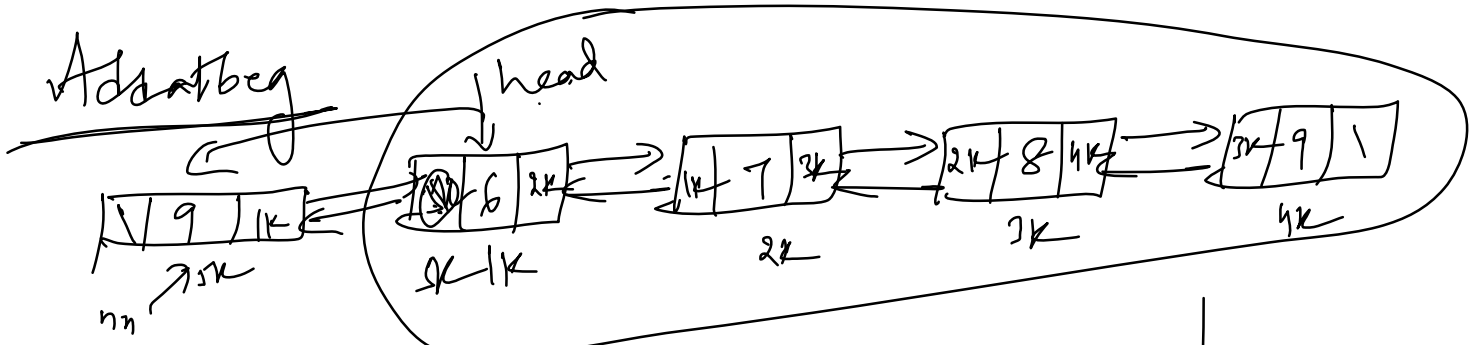

```
struct node *nn, *temp = head;
nn = (struct node *) malloc( sizeof(struct node));
nn -> data = x;
nn -> next = NULL;
if (head == NULL)
{
    head = nn;
    nn -> prev = NULL;
    return;
}
while(temp -> next != NULL)
    temp = temp -> next;
temp -> next = nn;
nn -> prev = temp;
```

Display:-




```
if(head == NULL)
{
    printf("List is empty\n");
    return;
}
```

```
while(temp != NULL)
{
printf("%d", temp->data);
    temp = temp->next;
}
```




```
nn=(struct node*)malloc(sizeof(struct node));
nn->data=x;
nn->prev=NULL;
```

```
nn->next=head;
if(head!=NULL)
head->prev=nn;
head=nn;
```